Machine Learning

Introduction - Linear Regression

Mohamed Berrada

Ecole Nationale Supérieure des Arts et Métiers de Meknès
Université Moulay Ismail
Filière GI-IADS

2021-2022

Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

Apprentissage automatique (Machine learning)

L'apprentissage automatique (en anglais**Machine Learning**) est un ensemble d'algorithmes qui permet à un ordinateur d'apprendre et de prédire des résultats sans être explicitement programmées.

Apprentissage automatique (Machine learning)

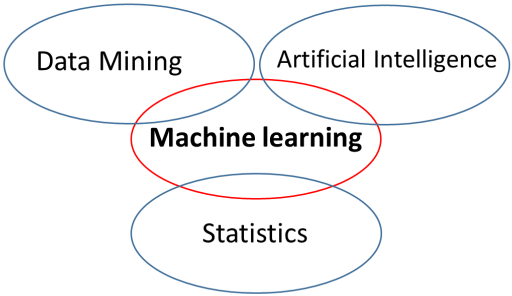
L'apprentissage automatique (en anglais**Machine Learning**) est un ensemble d'algorithmes qui permet à un ordinateur d'apprendre et de prédire des résultats sans être explicitement programmées.

Appelé aussi "**apprentissage artificiel**" ou "**apprentissage statistique**" du fait qu'il est fondé sur des approches mathématique et statistique, et qu'il est largement utilisé en intelligence artificiel.

Apprentissage automatique (Machine learning)

L'apprentissage automatique (en anglais**Machine Learning**) est un ensemble d'algorithmes qui permet à un ordinateur d'apprendre et de prédire des résultats sans être explicitement programmées.

Appelé aussi "**apprentissage artificiel**" ou "**apprentissage statistique**" du fait qu'il est fondé sur des approches mathématique et statistique, et qu'il est largement utilisé en intelligence artificiel.



Apprentissage automatique (Machine learning)

- Premier réseau de neurones artificiel :Warren Mc Culloch et Walter Pitts en 1943 décrivent le fonctionnement des neurones à l'aide de circuits électriques.

Apprentissage automatique (Machine learning)

- Premier réseau de neurones artificiel :Warren Mc Culloch et Walter Pitts en 1943 décrivent le fonctionnement des neurones à l'aide de circuits électriques.
- Années 90, le ML se distinct comme un domaine à part en se basant beaucoup plus sur des méthodes et modèles empruntés aux statistiques et à la théorie des probabilités.

Apprentissage automatique (Machine learning)

- Premier réseau de neurones artificiel :Warren Mc Culloch et Walter Pitts en 1943 décrivent le fonctionnement des neurones à l'aide de circuits électriques.
- Années 90, le ML se distinct comme un domaine à part en se basant beaucoup plus sur des méthodes et modèles empruntés aux statistiques et à la théorie des probabilités.
- Développement important des algorithmes de ML en bénéficiant de l'augmentation des capacités de calcul et de stockage des ordinateurs, et de l'omniprésence d'Internet et l'arrivée du big data

Apprentissage automatique (Machine learning)

- Premier réseau de neurones artificiel :Warren Mc Culloch et Walter Pitts en 1943 décrivent le fonctionnement des neurones à l'aide de circuits électriques.
- Années 90, le ML se distinct comme un domaine à part en se basant beaucoup plus sur des méthodes et modèles empruntés aux statistiques et à la théorie des probabilités.
- Développement important des algorithmes de ML en bénéficiant de l'augmentation des capacités de calcul et de stockage des ordinateurs, et de l'omniprésence d'Internet et l'arrivée du big data
- ML est largement utilisé en IA et en sciences des données : vision par ordinateur, traitement du langage naturel, reconnaissance vocale, diagnostic médical, tendances de consommation, auto-conduite, moteurs de recherche, etc.

Steps of Machine Learning:

Steps of Machine Learning:

- Data collection and preparation

Steps of Machine Learning:

- Data collection and preparation
- Choosing a model

Apprentissage automatique (Machine learning)

Steps of Machine Learning:

- Data collection and preparation
- Choosing a model
- Training the model

Apprentissage automatique (Machine learning)

Steps of Machine Learning:

- Data collection and preparation
- Choosing a model
- Training the model
- Evaluation and parameter tuning

Apprentissage automatique (Machine learning)

Steps of Machine Learning:

- Data collection and preparation
- Choosing a model
- Training the model
- Evaluation and parameter tuning
- Prediction

Data Collection and Preparation



- **Raw data**:
  Text, Number, Image, Audio, Video
- **Sources of Data**

  Open Data, Simulated Data, Experiment !

Data Collection and Preparation



- **Raw data**:
  Text, Number, Image, Audio, Video
- **Sources of Data**
  Open Data, Simulated Data, Experiment !

- **Real-world data:**:
  unorganized, missing, or noisy.
- **Data cleaning:**
  Missing values, outliers, incorrect types, ...
- **Scaling/Normalizing**

Data Collection and Preparation



- **Raw data**:
  Text, Number, Image, Audio, Video
- **Sources of Data**
  Open Data, Simulated Data, Experiment !

- **Real-world data:**:
  unorganized, missing, or noisy.
- **Data cleaning:**
  Missing values, outliers, incorrect types, ...
- **Scaling/Normalizing**

Data Collection and Preparation

- **Data structure:** Table of values arranged in rows and columns. Rows are cases (observations) and columns are variables.

Features (x)

	x1	x2	x3	x4
Obs-1				
Obs-2				
Obs-3				
Obs-4				

- Real estate : area, distance from city centre, floor number, ...
- Image : pixel=3 features (RGB)

  An image of 100x100 pixels in color contains 30000 features (too much!)

Data Collection and Preparation

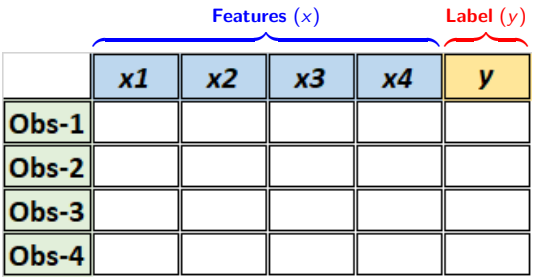
- **Data structure:** Table of values arranged in rows and columns. Rows are cases (observations) and columns are variables.

	Features (x)				Label (y)
	x1	*x2*	*x3*	*x4*	*y*
Obs-1					
Obs-2					
Obs-3					
Obs-4					

- Real estate : area, distance from city centre, floor number, ... + price (continuous)
- Image : pixel=3 features (RGB) + cat or dog (categorical)
  An image of 100x100 pixels in color contains 30000 features (too much!)

Data Collection and Preparation

- **Data structure:** Table of values arranged in rows and columns. Rows are cases (observations) and columns are variables.

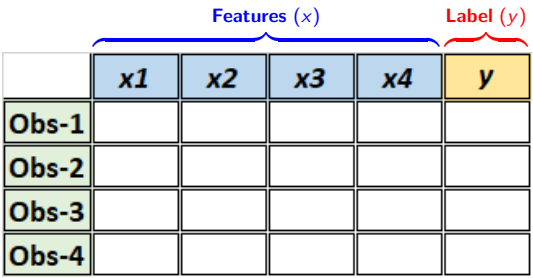


- Features
- Input variables
- Independent variables

- Real estate : area, distance from city centre, floor number, ... + price (continuous)
- Image : pixel=3 features (RGB) + cat or dog (categorical)
  An image of 100x100 pixels in color contains 30000 features (too much!)

Data Collection and Preparation

- **Data structure:** Table of values arranged in rows and columns. Rows are cases (observations) and columns are variables.



- Features
- Input variables
- Independent variables

- Target/Label
- Output variable
- Dependent variable

- Real estate : area, distance from city centre, floor number, ... + price (continuous)
- Image : pixel=3 features (RGB) + cat or dog (categorical)
  An image of 100x100 pixels in color contains 30000 features (too much!)

Data Collection and Preparation

Feature scaling

Scaling the features to make them comparable on a common scale

- Min-Max normalization

$$\tilde{x} = \frac{x - \min x}{\max x - \min x} \in [0, 1]$$

- Standard normalization

$$\tilde{x} = \frac{x - \bar{x}}{\sigma}$$

Data Collection and Preparation

Feature scaling

Scaling the features to make them comparable on a common scale

- Min-Max normalization

$$\tilde{x} = \frac{x - \min x}{\max x - \min x} \in [0, 1]$$

- Standard normalization

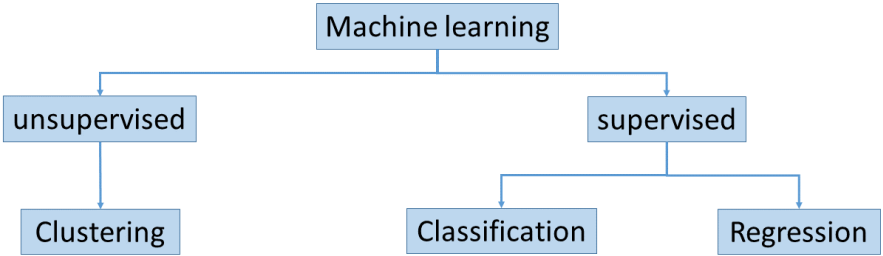
$$\tilde{x} = \frac{x - \bar{x}}{\sigma}$$

Why used feature scaling

- Feature scaling is essential for machine learning algorithms that calculate distances between data
- Gradient descent converges much faster with feature scaling
- It's important if regularization is used

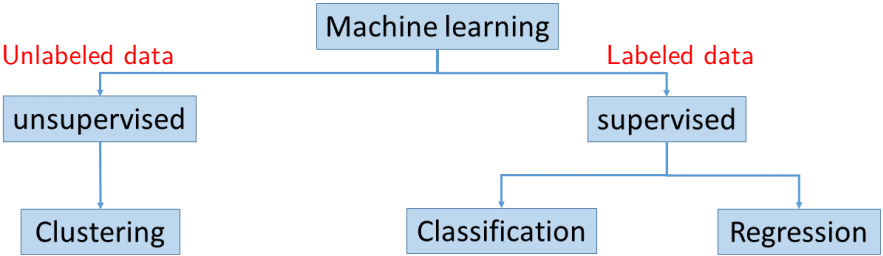
Choosing a model

Supervised vs unsupervised



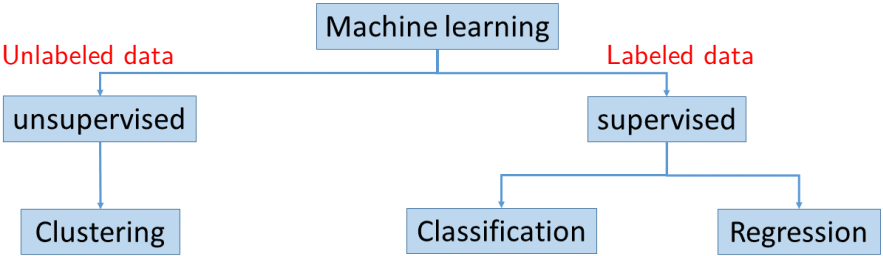
Choosing a model

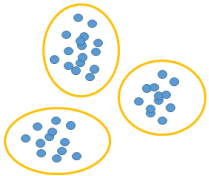
Supervised vs unsupervised



Choosing a model

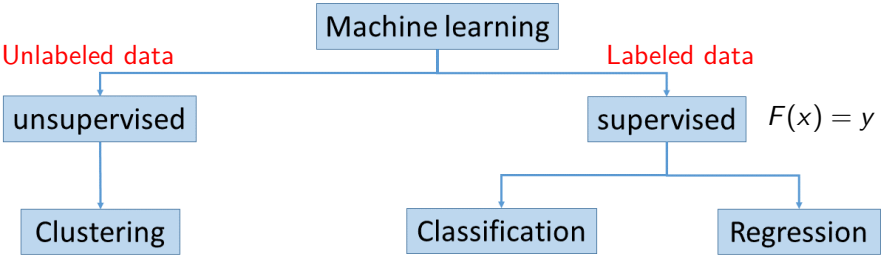
Supervised vs unsupervised

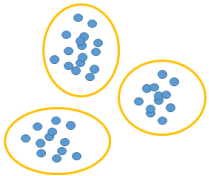




Choosing a model

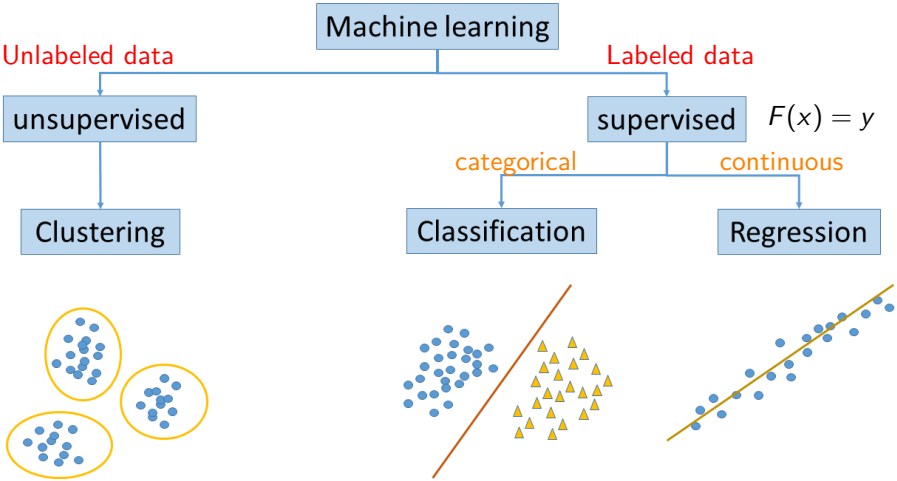
Supervised vs unsupervised





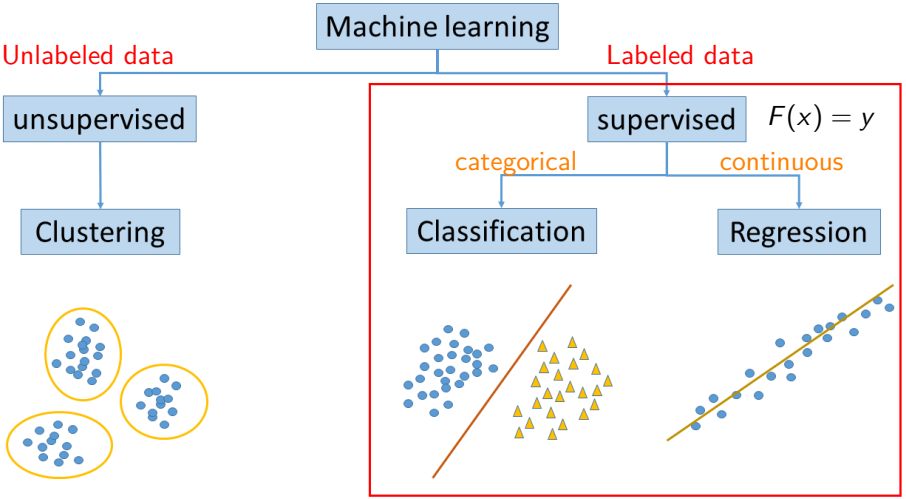
Choosing a model

Supervised vs unsupervised



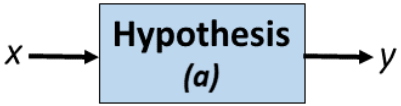
Choosing a model

Supervised vs unsupervised



Choosing a model (hypothesis)

Supervised model

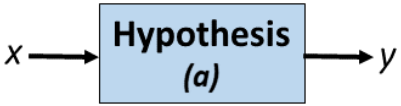


$F(x) = y$

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables

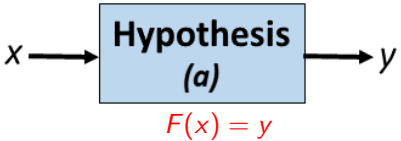


$F(x) = y$

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables

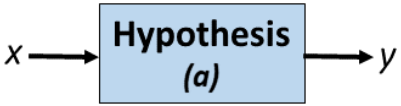


- Target
- Output variable
- Dependent variable

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



$F(x) = y$

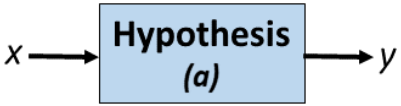
- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

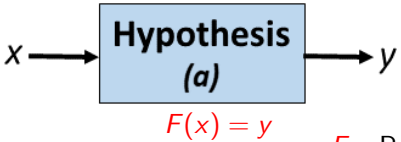
Choosing a model :

- $F(x) = a_0 + a_1 x$ (Linear regression)

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

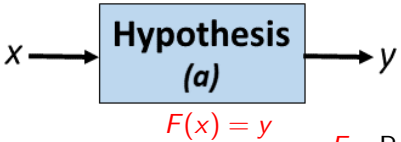
Choosing a model :

- $F(x) = a_0 + a_1 x$ (Linear regression)
- $F(x) = a_0 + a_1 x + a_2 x^2$ (polynomial)

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

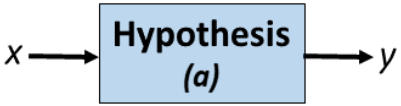
Choosing a model :

- $F(x) = a_0 + a_1 x$ (Linear regression)
- $F(x) = a_0 + a_1 x + a_2 x^2$ (polynomial)
- $F(x) = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_3$ (multivariable)

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

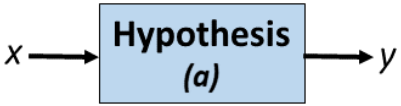
Choosing a model :

- $F(x) = a_0 + a_1 x$ (Linear regression)
- $F(x) = a_0 + a_1 x + a_2 x^2$ (polynomial)
- $F(x) = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_3$ (multivariable)
- $F(x) = \frac{1}{1+e^{-(a_0+a_1 x)}}$ (classification)

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

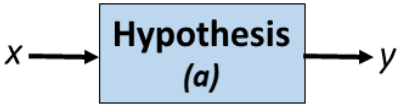
Choosing a model :

- $F(x) = a_0 + a_1 x$ (Linear regression)
- $F(x) = a_0 + a_1 x + a_2 x^2$ (polynomial)
- $F(x) = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_3$ (multivariable)
- $F(x) = \frac{1}{1+e^{-(a_0+a_1 x)}}$ (classification)
- ANN, SVM, Random Forest, CNN, $\cdots$

Choosing a model (hypothesis)

Supervised model

- Features
- Input variables
- Independent variables



- Target
- Output variable
- Dependent variable

F : Predictive/Decision model

Choosing a model :

- $F(x) = a_0 + a_1 x$ (Linear regression)
- $F(x) = a_0 + a_1 x + a_2 x^2$ (polynomial)
- $F(x) = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_3$ (multivariable)
- $F(x) = \frac{1}{1+e^{-(a_0+a_1 x)}}$ (classification)
- ANN, SVM, Random Forest, CNN, $\cdots$

$a = (a_0, a_1, \cdots, a_p)^t$ **the model weights**

Training the model

Training set

L'ensemble d'apprentissage est un ensemble de couple entrée-sortie $\mathcal{A} = \{(x(i), y(i)),\ i = 1 \cdots, m\}$ que nous observons et que nous utilisons pour déterminer les poids du modèle.

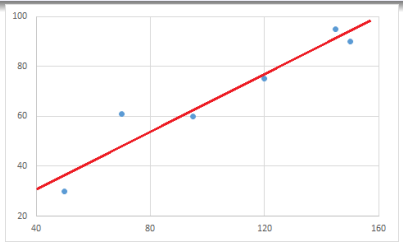
	A	B
1	superficie (x)	prix k$ (y)
2	50	30
3	150	90
4	70	61
5	95	60
6	120	75
7	145	95

Training the model

Training set

L'ensemble d'apprentissage est un ensemble de couple entrée-sortie $\mathcal{A} = \{(x(i), y(i)),\ i = 1 \cdots, m\}$ que nous observons et que nous utilisons pour déterminer les poids du modèle.

	A	B
1	superficie (x)	prix k$ (y)
2	50	30
3	150	90
4	70	61
5	95	60
6	120	75
7	145	95



Training

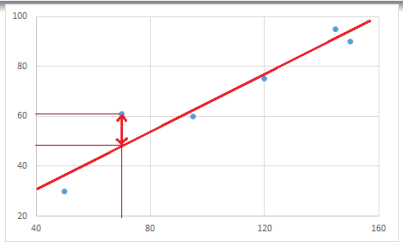
L'apprentissage (Training) est le processus permettant de déterminer les poids a tels que les écarts entre la sortie estimée $F(x(i))$ et la sortie observée $y(i)$ soient les plus petit possible.

Training the model

Training set

L'ensemble d'apprentissage est un ensemble de couple entrée-sortie $\mathcal{A} = \{(x(i), y(i)),\ i = 1 \cdots, m\}$ que nous observons et que nous utilisons pour déterminer les poids du modèle.

	A	B
1	superficie (x)	prix k$ (y)
2	50	30
3	150	90
4	70	61
5	95	60
6	120	75
7	145	95



Training

L'apprentissage (Training) est le processus permettant de déterminer les poids a tels que les écarts entre la sortie estimée $F(x(i))$ et la sortie observée $y(i)$ soient les plus petit possible.

Evaluating a regression model

- Root Mean Square Error (RMSE) required to be small

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(F(x(i)) - y(i))^2}$$

Evaluating a regression model

- Root Mean Square Error (RMSE) required to be small

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(F(x(i)) - y(i))^2}$$

Example: An error of 10\$ in prediction of both DVD and car prices:
$\begin{cases} obs\ DVD\ price = 4\$ \\ pred\ DVD\ price = 14\$ \end{cases}$ and $\begin{cases} obs\ car\ price = 8000\$ \\ pred\ car\ price = 8010\$ \end{cases}$
gives the same $MSE = (F(x) - y)^2 = 10^2 = 100$

Evaluating a regression model

- Root Mean Square Error (RMSE) required to be small

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(F(x(i)) - y(i))^2}$$

Example: An error of 10$ in prediction of both DVD and car prices:
$\begin{cases} obs\ DVD\ price = 4\$ \\ pred\ DVD\ price = 14\$ \end{cases}$ and $\begin{cases} obs\ car\ price = 8000\$ \\ pred\ car\ price = 8010\$ \end{cases}$
gives the same $MSE = (F(x) - y)^2 = 10^2 = 100$

- Relative Squared Error (RSE)

$$RSE = \frac{\sum_{i=1}^{m}(F(x(i)) - y(i))^2}{\sum_{i=1}^{m}(\overline{y} - y(i))^2}$$

$\overline{y} = \frac{1}{m}\sum_{i=1}^{m} y(i)$.

Evaluating a regression model

- Root Mean Square Error (RMSE) required to be small

$$RMSE = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(F(x(i)) - y(i))^2}$$

Example: An error of 10\$ in prediction of both DVD and car prices:
$\begin{cases} obs\ DVD\ price = 4\$ \\ pred\ DVD\ price = 14\$ \end{cases}$ and $\begin{cases} obs\ car\ price = 8000\$ \\ pred\ car\ price = 8010\$ \end{cases}$
gives the same $MSE = (F(x) - y)^2 = 10^2 = 100$

- Relative Squared Error (RSE)

$$RSE = \frac{\sum_{i=1}^{m}(F(x(i)) - y(i))^2}{\sum_{i=1}^{m}(\overline{y} - y(i))^2}$$

$\overline{y} = \frac{1}{m}\sum_{i=1}^{m} y(i)$.

- Coefficient of determination ($R^2 \in [0, 1]$) required to be close to 1

$$R^2 = 1 - RSE$$

Generalization

Généralisation

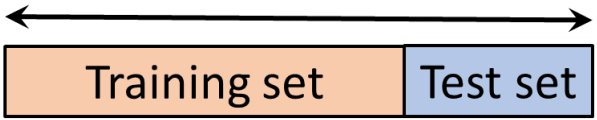
Capacité du modèle à faire de bonnes prédictions pour de nouvelles observations.

Generalization

Généralisation

Capacité du modèle à faire de bonnes prédictions pour de nouvelles observations.

- **Data splitting**

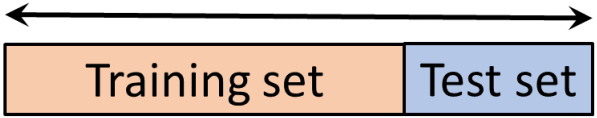


Generalization

Généralisation

Capacité du modèle à faire de bonnes prédictions pour de nouvelles observations.

- **Data splitting**



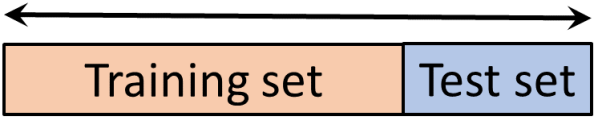
- **Training set:** pour trouver les poids a

Generalization

Généralisation

Capacité du modèle à faire de bonnes prédictions pour de nouvelles observations.

- **Data splitting**

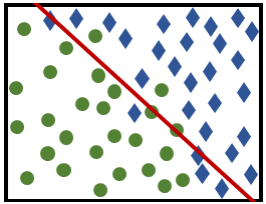


- **Training set:** pour trouver les poids *a*
- **Test set:** pour s'assurer que le modèle généralise

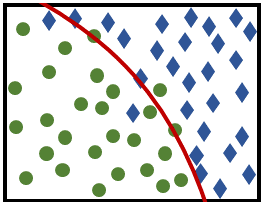
Generalization

Biais vs variance

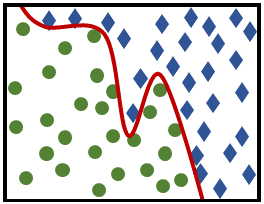
Underfitting



Balanced



Overfitting



Generalization

Biais vs variance



Generalization

Biais vs variance

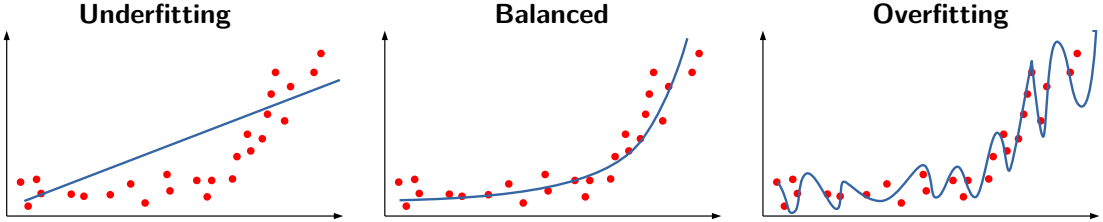


Biais

Hypothèse erronée dans l'algorithme d'apprentissage. Un biais élevé correspond à une mauvaise représentation de la relation entre la sortie y et l'entrée x. Il conduit à un problème d'underfitting.

Generalization

Biais vs variance



Biais

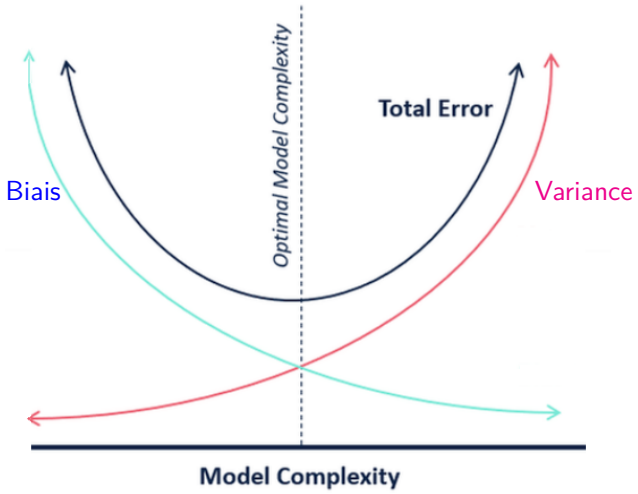
Hypothèse erronée dans l'algorithme d'apprentissage. Un biais élevé correspond à une mauvaise représentation de la relation entre la sortie y et l'entrée x. Il conduit à un problème d'underfitting.

Variance

Erreur due à la sensibilité du modèle au bruit. Le modèle colle trop aux données d'apprentissage. Une variance forte est synonyme d'un problème d'overfitting.

Generalization

Biais vs variance



The tradeoff:

- Dimensionality reduction
- Select the best model
- Regularization
- Ensemble Methods

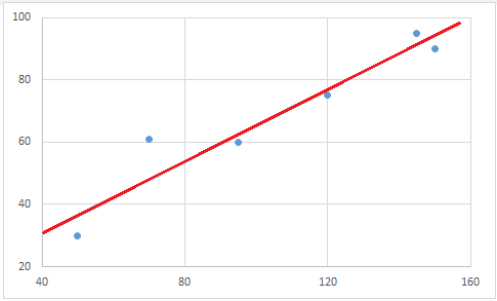
Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

Régression linéaire

Ensemble d'apprentissage (Training set)

	A	B
1	superficie (x)	prix k$ (y)
2	50	30
3	150	90
4	70	61
5	95	60
6	120	75
7	145	95



Hypothèse

$$F(x) = a_0 + a_1 x,$$

m : le nombre d'individus dans l'ensemble d'apprentissage

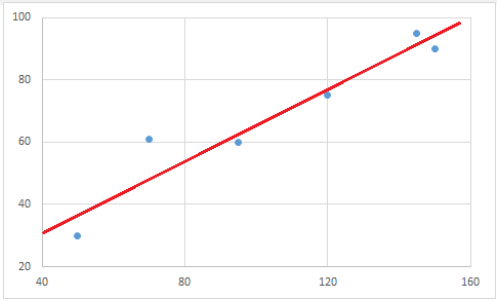
x : variable d'entrée / variable indépendante / features

y : variable de sortie / variable dépendante / target

Régression linéaire

Ensemble d'apprentissage (Training set)

	A	B
1	superficie (x)	prix k$ (y)
2	50	30
3	150	90
4	70	61
5	95	60
6	120	75
7	145	95



Hypothèse

$$F(x) = a_0 + a_1 x,$$

m : le nombre d'individus dans l'ensemble d'apprentissage

x : variable d'entrée / variable indépendante / features

y : variable de sortie / variable dépendante / target

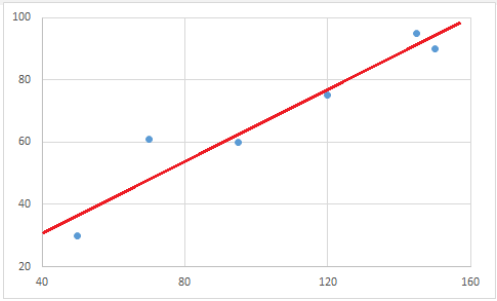
$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = \sum_{i=0}^{p} a_i x_i, \ (x_0 = 1) = x^T a.$$

$$x = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{pmatrix} \text{ et } a = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_p \end{pmatrix}$$

Régression linéaire

Ensemble d'apprentissage (Training set)

	A	B
1	superficie (x)	prix k$ (y)
2	50	30
3	150	90
4	70	61
5	95	60
6	120	75
7	145	95



Hypothèse

$$F(x) = a_0 + a_1 x,$$

m : le nombre d'individus dans l'ensemble d'apprentissage

x : variable d'entrée / variable indépendante / features

y : variable de sortie / variable dépendante / target

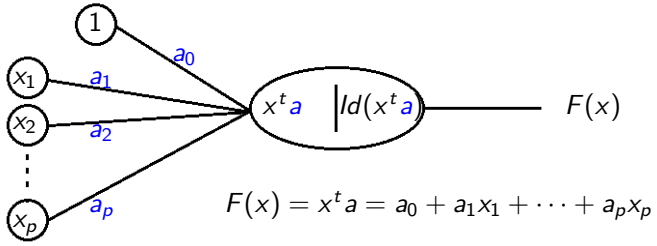
$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = \sum_{i=0}^{p} a_i x_i, \ (x_0 = 1) = x^T a.$$

$$x = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{pmatrix} \text{ et } a = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_p \end{pmatrix}$$

Régression linéaire simple si $p = 1$, et multiple si $p > 1$.

Régression linéaire

- Schématisation de la régression linéaire



Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

Cost function - Mean square loss

Méthode des moindres carrées

- Modèle linéaire $F : \mathbb{R}^p \to \mathbb{R}$,

$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = \sum_{i=0}^{p} a_i x_i, \ (x_0 = 1)$$

Cost function - Mean square loss

Méthode des moindres carrées

- Modèle linéaire $F : \mathbb{R}^p \to \mathbb{R}$,

$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = \sum_{i=0}^{p} a_i x_i, \ (x_0 = 1)$$

- Minimiser les écarts entre les sorties estimées $F(x(i))$ et les sorties observées $y(i)$ pour $i = 1, \cdots, m$:

$$J(a) = \frac{1}{2m} \sum_{i=1}^{m} (F(x(i)) - y(i))^2 \qquad \textbf{Mean square loss}$$

La division par $2m$ est pour normaliser.

Cost function - Mean square loss

Méthode des moindres carrées

- Modèle linéaire $F : \mathbb{R}^p \to \mathbb{R}$,

$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = \sum_{i=0}^{p} a_i x_i, \ (x_0 = 1)$$

- Minimiser les écarts entre les sorties estimées $F(x(i))$ et les sorties observées $y(i)$ pour $i = 1, \cdots, m$:

$$J(a) = \frac{1}{2m} \sum_{i=1}^{m} (F(x(i)) - y(i))^2 \qquad \textbf{Mean square loss}$$

La division par $2m$ est pour normaliser.

- On écrit aussi

$$J(a) = \sum_{i=1}^{m} J_i(a) \text{ où } J_i(a) = \frac{(F(x(i)) - y(i))^2}{2m}.$$

Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

Normal equations

- Écriture matricielle : $Y = \begin{pmatrix} y(1) \\ \vdots \\ y(m) \end{pmatrix}$ et $X = \begin{pmatrix} 1 & x_1(1) & \cdots & x_p(1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1(m) & \cdots & x_p(m) \end{pmatrix}$

Normal equations

- Écriture matricielle : $Y = \begin{pmatrix} y(1) \\ \vdots \\ y(m) \end{pmatrix}$ et $X = \begin{pmatrix} 1 & x_1(1) & \cdots & x_p(1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1(m) & \cdots & x_p(m) \end{pmatrix}$
- La fonction coût s'écrit alors

$$J(a) = \frac{1}{2m}(Xa - Y)^t(Xa - Y)$$

Normal equations

- Écriture matricielle : $Y = \begin{pmatrix} y(1) \\ \vdots \\ y(m) \end{pmatrix}$ et $X = \begin{pmatrix} 1 & x_1(1) & \cdots & x_p(1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1(m) & \cdots & x_p(m) \end{pmatrix}$
- La fonction coût s'écrit alors

$$J(a) = \frac{1}{2m}(Xa - Y)^t(Xa - Y)$$

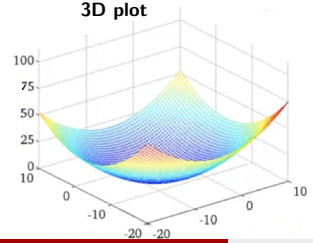
◇ J est différentiable $\quad \nabla_a J = \frac{1}{m} X^t(Xa - Y)$

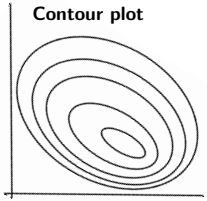
Normal equations

- Écriture matricielle : $Y = \begin{pmatrix} y(1) \\ \vdots \\ y(m) \end{pmatrix}$ et $X = \begin{pmatrix} 1 & x_1(1) & \cdots & x_p(1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1(m) & \cdots & x_p(m) \end{pmatrix}$
- La fonction coût s'écrit alors

$$J(a) = \frac{1}{2m}(Xa - Y)^t(Xa - Y)$$

- ◇ J est différentiable $\quad \nabla_a J = \frac{1}{m} X^t(Xa - Y)$
- ◇ J est convexe





Normal equations

- Le Minimum de J atteint si $\nabla_a J = 0$, c'est à dire
$$\frac{1}{m} X^t (Xa - Y) = 0$$

Normal equations

- Le Minimum de J atteint si $\nabla_a J = 0$, c'est à dire
$$\frac{1}{m} X^t (Xa - Y) = 0$$
- On obtient alors le système des **équations normales**
$$X^t X a = X^t Y$$
où $X^t X$ est une matrice de dimension $p \times p$.

Normal equations

- Le Minimum de J atteint si $\nabla_a J = 0$, c'est à dire
$$\frac{1}{m} X^t (Xa - Y) = 0$$
- On obtient alors le système des **équations normales**
$$X^t X a = X^t Y$$
où $X^t X$ est une matrice de dimension $p \times p$.
- Les poids optimaux exacts
$$a = \left(X^t X\right)^{-1} X^t Y.$$

Normal equations

- Le Minimum de J atteint si $\nabla_a J = 0$, c'est à dire
$$\frac{1}{m} X^t (Xa - Y) = 0$$
- On obtient alors le système des **équations normales**
$$X^t X a = X^t Y$$
où $X^t X$ est une matrice de dimension $p \times p$.
- Les poids optimaux exacts
$$a = \left(X^t X\right)^{-1} X^t Y.$$

age	hauteur
10	52
12	60
15	73
21	110

Normal equations

- Le Minimum de J atteint si $\nabla_a J = 0$, c'est à dire
$$\frac{1}{m} X^t (Xa - Y) = 0$$
- On obtient alors le système des **équations normales**
$$X^t X a = X^t Y$$
où $X^t X$ est une matrice de dimension $p \times p$.
- Les poids optimaux exacts
$$a = \left(X^t X\right)^{-1} X^t Y.$$

age	hauteur
10	52
12	60
15	73
21	110

$$\begin{cases} 4a_0 + 58a_1 = 295 \\ 58a_0 + 910a_1 = 4645 \end{cases}$$

Normal equations

- Le Minimum de J atteint si $\nabla_a J = 0$, c'est à dire
$$\frac{1}{m} X^t (Xa - Y) = 0$$
- On obtient alors le système des **équations normales**
$$X^t X a = X^t Y$$
où $X^t X$ est une matrice de dimension $p \times p$.
- Les poids optimaux exacts
$$a = \left(X^t X\right)^{-1} X^t Y.$$

age	hauteur
10	52
12	60
15	73
21	110

$$\begin{cases} 4a_0 + 58a_1 = 295 \\ 58a_0 + 910a_1 = 4645 \end{cases}$$

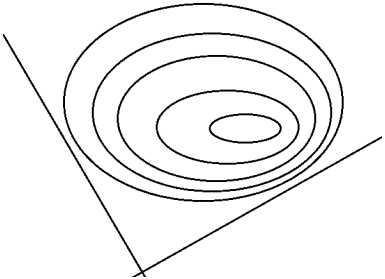
- $X^t X$ peut être non inversible!
- Son inversion devient très couteuse si le nombre de features p est très grand!
- Méthode non pratique pour d'autre modèles ... !

Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

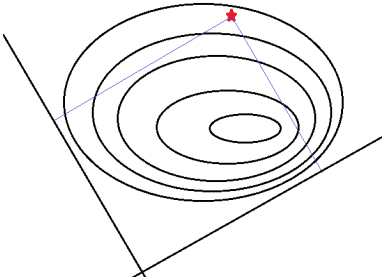
Gradient descent

- **Gradient descent (Batch GD):**



Gradient descent

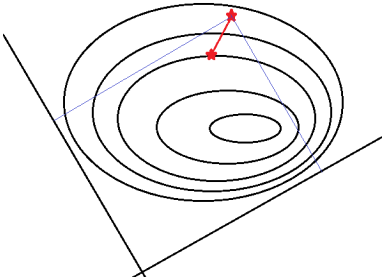
- **Gradient descent (Batch GD):**
  - Choisir des valeurs de départ pour a_0 et a_1



Gradient descent

- **Gradient descent (Batch GD):**
  - Choisir des valeurs de départ pour a_0 et a_1
  - Répéter jusqu'à convergence :
    - $a = a - \alpha \nabla_a J$

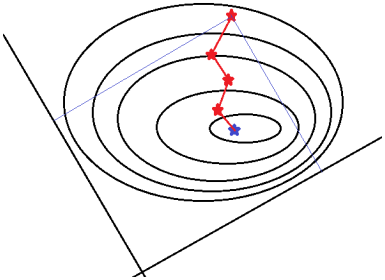
      where α : learning rate and $\nabla_a J = \sum_{i=1}^{m} \nabla_a J_i$



Gradient descent

- **Gradient descent (Batch GD):**
  - Choisir des valeurs de départ pour a_0 et a_1
  - Répéter jusqu'à convergence :
    - $a = a - \alpha \nabla_a J$

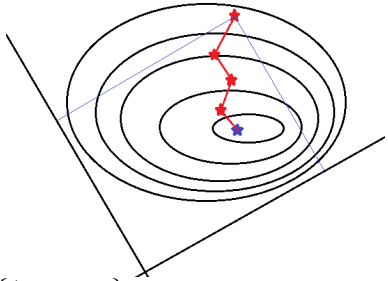
      where α : learning rate and $\nabla_a J = \sum_{i=1}^{m} \nabla_a J_i$



Gradient descent

- **Gradient descent (Batch GD):**
  - Choisir des valeurs de départ pour a_0 et a_1
  - Répéter jusqu'à convergence :
    - $a = a - \alpha \nabla_a J$

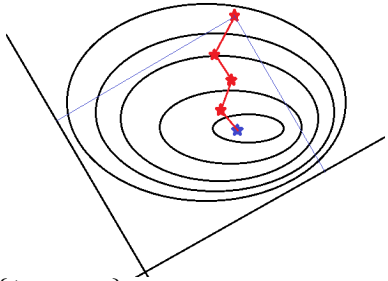
    where α : learning rate and $\nabla_a J = \sum_{i=1}^{m} \nabla_a J_i$



- **Stochastic Gradient descent:** i is randomly chosen from $\{1, \cdots, m\}$

  $\nabla_a J \approx \nabla_a J_i$

Gradient descent



- **Gradient descent (Batch GD):**
  - Choisir des valeurs de départ pour a_0 et a_1
  - Répéter jusqu'à convergence :
    - $a = a - \alpha \nabla_a J$

    where α : learning rate and $\nabla_a J = \sum_{i=1}^{m} \nabla_a J_i$
- **Stochastic Gradient descent:** i is randomly chosen from $\{1, \cdots, m\}$

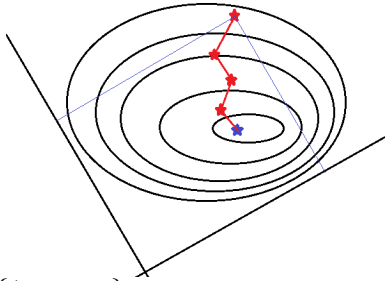
  $\nabla_a J \approx \nabla_a J_i$
- **Mini-Batch Gradient descent:** $I \subset \{1, \cdots, m\}$ randomly chosen

  $\nabla_a J \approx \sum_{i \in I} \nabla_a J_i$

Gradient descent

- **Gradient descent (Batch GD):**
  - Choisir des valeurs de départ pour a_0 et a_1
  - Répéter jusqu'à convergence :
    - $a = a - \alpha \nabla_a J$

    where α : learning rate and $\nabla_a J = \sum_{i=1}^{m} \nabla_a J_i$



- **Stochastic Gradient descent:** i is randomly chosen from $\{1, \cdots, m\}$

  $\nabla_a J \approx \nabla_a J_i$

- **Mini-Batch Gradient descent:** $I \subset \{1, \cdots, m\}$ randomly chosen

  $\nabla_a J \approx \sum_{i \in I} \nabla_a J_i$

- **Cas générale d'un algorithme itératif :**

  $a = a + \alpha_k d_k$.

Gradient descent

Gradient descent algorithm

- initialization: a, α, epochs, ε,
- $n = 0$, $d = \nabla_a J$ et $ng = \|d\|$
- While $ng > \varepsilon$ and $n <$epochs
  - $a = a - \alpha \times d$
  - $n = n + 1$, $d = \nabla_a J$ et $ng = \|d\|$
- return a

Gradient descent

- **Choosing the learning rate:**

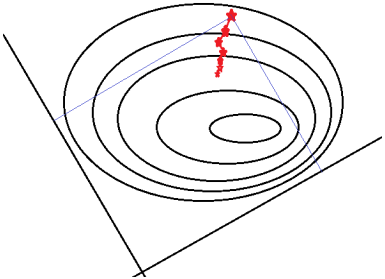
$$a = a - \alpha \nabla_a J$$

Gradient descent

- **Choosing the learning rate:**

$$a = a - \alpha \nabla_a J$$

Small learning rate (Slow convergence)

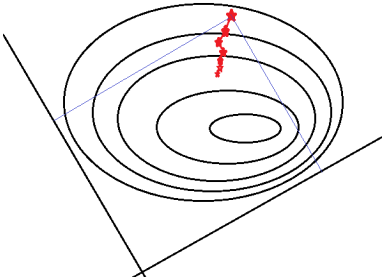


Gradient descent

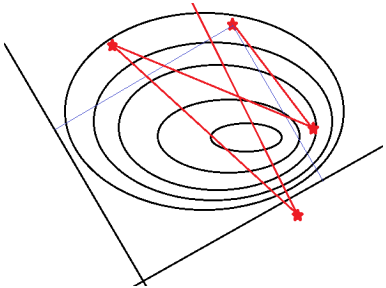
- **Choosing the learning rate:**

$$a = a - \alpha \nabla_a J$$

Small learning rate (Slow convergence)



Big learning rate (May not converge)



Sommaire

1. Définition et introduction
2. Régression linéaire
   - Cost function
   - Normal equations
   - Gradient descent
3. Maximisation de la vraisemblance

Maximisation de la vraisemblance

- La régression linéaire

$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = x^t a \quad \text{avec } x = \begin{pmatrix} 1 \\ x_1 \\ . \\ . \\ x_p \end{pmatrix} \text{ et } a = \begin{pmatrix} a_0 \\ a_1 \\ . \\ . \\ a_p \end{pmatrix}$$

Maximisation de la vraisemblance

- La régression linéaire

$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = x^t a \quad \text{avec } x = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_p \end{pmatrix} \text{ et } a = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_p \end{pmatrix}$$

- Chercher les poids a pour lesquels il est plus vraisemblable d'observer les données $\mathbb{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$
  - $x(i)$ est une réalisation d'une variable aléatoire x ($p-$dimension)
  - $y(i)$ est une réalisation d'une variable aléatoire y

Maximisation de la vraisemblance

- La régression linéaire

$$F(x) = a_0 + a_1 x_1 + \cdots + a_p x_p = x^t a \quad \text{avec } x = \begin{pmatrix} 1 \\ x_1 \\ . \\ \vdots \\ x_p \end{pmatrix} \text{ et } a = \begin{pmatrix} a_0 \\ a_1 \\ . \\ \vdots \\ a_p \end{pmatrix}$$

- Chercher les poids a pour lesquels il est plus vraisemblable d'observer les données $\mathbb{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$
  - $x(i)$ est une réalisation d'une variable aléatoire x ($p-$dimension)
  - $y(i)$ est une réalisation d'une variable aléatoire y
- Les hypothèses
  - Linéarité $y(i) = a_0 + a_1 x_1(i) + \cdots + a_p x_p(i) + \varepsilon_i = x(i)^t a + \varepsilon_i$
  - $+\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$
  - Les couples $(x(i), y(i))$ sont indépendantes les unes des autres
    Il est claire, dans ce cas, que $p(y/x; a) \sim \mathcal{N}(x^t a, \sigma^2)$

Maximisation de la vraisemblance

- Le but est alors de maximiser, par rapport aux poids a, la probabilité

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right]$$

qu'on appelle la vraisemblance (likelihood)

Maximisation de la vraisemblance

- Le but est alors de maximiser, par rapport aux poids a, la probabilité

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right]$$

qu'on appelle la vraisemblance (likelihood)

- D'après les hypothèses, on a

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right] = \prod_{i=1}^{m} p((x(i), y(i))/a)$$

Maximisation de la vraisemblance

- Le but est alors de maximiser, par rapport aux poids a, la probabilité

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right]$$

qu'on appelle la vraisemblance (likelihood)

- D'après les hypothèses, on a

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right] = \prod_{i=1}^{m} p((x(i), y(i))/a)$$

- La relation de Bayes nous permet d'écrire

$$\prod_{i=1}^{m} p((x(i), y(i))/a) = \prod_{i=1}^{m} p(y(i)/x(i), a)p(x(i))$$

$p(x(i))$ peux être assimilé à une constante puisqu'il ne dépend pas de a.

Maximisation de la vraisemblance

- Le but est alors de maximiser, par rapport aux poids a, la probabilité

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right]$$

qu'on appelle la vraisemblance (likelihood)

- D'après les hypothèses, on a

$$p\left[(x(1), y(1)), \cdots, (x(m), y(m))/a\right] = \prod_{i=1}^{m} p((x(i), y(i))/a)$$

- La relation de Bayes nous permet d'écrire

$$\prod_{i=1}^{m} p((x(i), y(i))/a) = \prod_{i=1}^{m} p(y(i)/x(i), a)p(x(i))$$

$p(x(i))$ peux être assimilé à une constante puisqu'il ne dépend pas de a.

- On trouve alors le problème dit de maximisation de la vraisemblance (likelihood maximization)

$$\arg\max_{a} \prod_{i=1}^{m} p(y(i)/x(i), a)$$

Maximisation de la vraisemblance

- On applique la fonction log, on trouve alors le problème de la Log-vraisemblance (log-likelihood)

$$\arg\max_{a} \sum_{i=1}^{m} \ln(p(y(i)/x(i), a))$$

Maximisation de la vraisemblance

- On applique la fonction log, on trouve alors le problème de la Log-vraisemblance (log-likelihood)

$$\arg\max_{a} \sum_{i=1}^{m} \ln(p(y(i)/x(i), a))$$

- Commme $p(y/x; a) \sim \mathcal{N}(F(x), \sigma^2)$ donc $p(y(i)/x(i), a) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{[F(x(i))-y(i)]^2}{2\sigma^2}\right)$

Maximisation de la vraisemblance

- On applique la fonction log, on trouve alors le problème de la Log-vraisemblance (log-likelihood)

$$\arg\max_a \sum_{i=1}^{m} \ln(p(y(i)/x(i), a))$$

- Commme $p(y/x; a) \sim \mathcal{N}(F(x), \sigma^2)$ donc $p(y(i)/x(i), a) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{[F(x(i))-y(i)]^2}{2\sigma^2}\right)$
- Le problème devient

$$\arg\max_a \left(\sum_{i=1}^{m} -\frac{[F(x(i)) - y(i)]^2}{2\sigma^2} - \ln\left(\sqrt{2\pi}\sigma\right)\right)$$

Maximisation de la vraisemblance

- On applique la fonction log, on trouve alors le problème de la Log-vraisemblance (log-likelihood)

$$\arg\max_a \sum_{i=1}^{m} \ln(p(y(i)/x(i), a))$$

- Commme $p(y/x; a) \sim \mathcal{N}(F(x), \sigma^2)$ donc $p(y(i)/x(i), a) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{[F(x(i))-y(i)]^2}{2\sigma^2}\right)$
- Le problème devient

$$\arg\max_a \left(\sum_{i=1}^{m} -\frac{[F(x(i)) - y(i)]^2}{2\sigma^2} - \ln\left(\sqrt{2\pi}\sigma\right)\right)$$

- On retrouve la méthode des moindres carrés

$$\arg\min_a \left(\sum_{i=1}^{m} (F(x(i)) - y(i))^2\right)$$

Objectifs du TP N° 1

1. Obtenir les données pour la régression
   1. Utiliser la librairie `pandas` pour lire les données à partir d'un fichier CSV
   2. Simulé des données par `make_regression` du module `sklearn.datasets`
2. Mettre en œuvre le modèle de régression linéaire par
   1. Equations normales
   2. Descente de gradient (effet du learning rate)
   3. `LinearRegression` du module `sklearn.linear_model`
3. Faire des figures dans le cas 1D
4. Evaluer la qualité du modèle de régression par `mean_squared_error` et `r2_score` du module `sklearn.metrics`
5. Séparer les données en training set et test set par `train_test_split` du module `sklearn.model_selection`

End Lecture N° 1

Machine Learning

Classification

Mohamed Berrada

Ecole Nationale Supérieure des Arts et Métiers de Meknès
Université Moulay Ismail
Filière GI-IADS

2020-2021

Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

Classification binaire

	Features (x)				Label (y)
	x1	x2	x3	x4	y
Obs-1					
Obs-2					
Obs-3					
Obs-4					

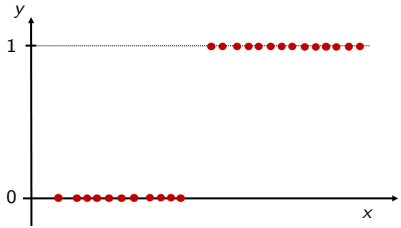
Classification binaire

	Features (x)				Label (y)
	x1	x2	x3	x4	y
Obs-1					0
Obs-2					1
Obs-3					1
Obs-4					0

- La variable dépendante y prend les deux valeurs $\{0, 1\}$:
  - Email : Spams ou non spams
  - Marketing : Client habituel ou occasionnel
  - Banque : Transaction frauduleuse ou régulière
  - ...

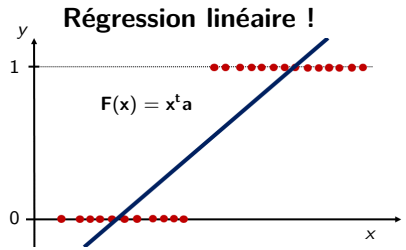
Régression logistique

- La régression logistique est une méthode de classification supervisée binaire.



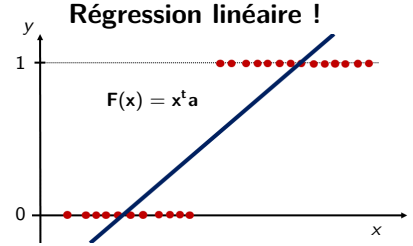
Régression logistique

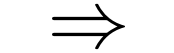
- La régression logistique est une méthode de classification supervisée binaire.



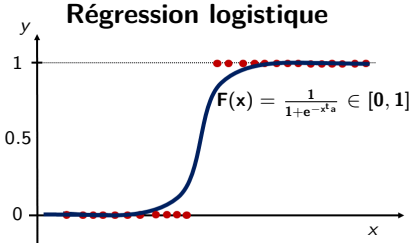
Régression logistique

- La régression logistique est une méthode de classification supervisée binaire.



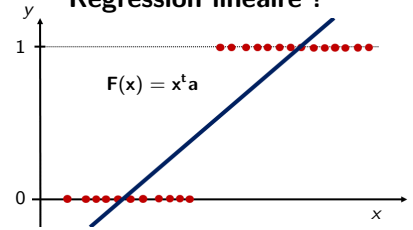


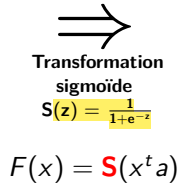
$F(x) = \mathbf{S}(x^t a)$

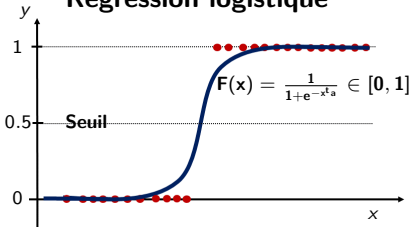


Régression logistique

- La régression logistique est une méthode de classification supervisée binaire.





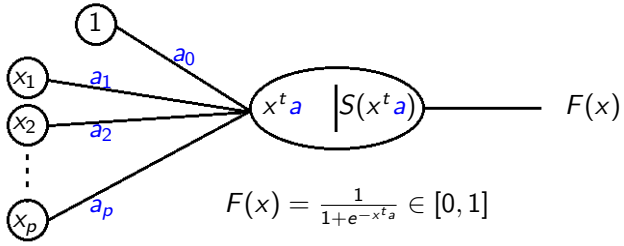


Seuil d'affectation

- Si $\mathbf{F(x) \geq 0.5}$ alors x est affecté à 1
- Si $\mathbf{F(x) < 0.5}$ alors x est affecté à 0

Régression logistique

- Schématisation de la régression logistique



Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

Cost function

Likelihood

- Soit $\mathcal{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$ l'ensemble d'apprentissage

Cost function

Likelihood

- Soit $\mathcal{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$ l'ensemble d'apprentissage
- Le but est d'avoir un modèle telle que $F(x(i))$ soit le plus proche possible de $y(i)$

Cost function

Likelihood

- Soit $\mathcal{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$ l'ensemble d'apprentissage
- Le but est d'avoir un modèle telle que $F(x(i))$ soit le plus proche possible de $y(i)$
- La sortie de modèle de RL est $F(x) = \frac{1}{1+e^{-x^t a}} \in [0, 1]$

Cost function

Likelihood

- Soit $\mathcal{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$ l'ensemble d'apprentissage
- Le but est d'avoir un modèle telle que $F(x(i))$ soit le plus proche possible de $y(i)$
- La sortie de modèle de RL est $F(x) = \frac{1}{1+e^{-x^t a}} \in [0, 1]$
- $F(x)$ pourra être vue comme une probabilité d'appartenance à la classe $y = 1$. On écrit
  - $p(y = 1/x, a) = F(x)$
  - $p(y = 0/x, a) = 1 - F(x)$

Cost function

Likelihood

- Soit $\mathcal{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$ l'ensemble d'apprentissage
- Le but est d'avoir un modèle telle que $F(x(i))$ soit le plus proche possible de $y(i)$
- La sortie de modèle de RL est $F(x) = \frac{1}{1+e^{-x^t a}} \in [0, 1]$
- $F(x)$ pourra être vue comme une probabilité d'appartenance à la classe $y = 1$. On écrit
  - $p(y = 1/x, a) = F(x)$
  - $p(y = 0/x, a) = 1 - F(x)$
- Pour les données d'apprentissage, on a la probabilité
$$p(y = y(i)/x(i), a) = F(x(i))^{y(i)}(1 - F(x(i)))^{1-y(i)}$$
qui doit être maximale (proche de 1) pour chaque $i = 1, \cdots, m$.

Cost function

Likelihood

- Soit $\mathcal{A} = \{(x(i), y(i)),\ i = 1, \cdots, m\}$ l'ensemble d'apprentissage
- Le but est d'avoir un modèle telle que $F(x(i))$ soit le plus proche possible de $y(i)$
- La sortie de modèle de RL est $F(x) = \frac{1}{1+e^{-x^t a}} \in [0, 1]$
- $F(x)$ pourra être vue comme une probabilité d'appartenance à la classe $y = 1$. On écrit
  - $p(y = 1/x, a) = F(x)$
  - $p(y = 0/x, a) = 1 - F(x)$
- Pour les données d'apprentissage, on a la probabilité
$$p(y = y(i)/x(i), a) = F(x(i))^{y(i)}(1 - F(x(i)))^{1-y(i)}$$
qui doit être maximale (proche de 1) pour chaque $i = 1, \cdots, m$.
- La fonction vraisemblance qu'on doit alors maximiser par rapport aux poids est
$$L(a) = \prod_{i=1}^{m} p(y = y(i)/x(i), a) = \prod_{i=1}^{m} F(x(i))^{y(i)}(1 - F(x(i)))^{1-y(i)}$$

Cost function - Cross-Entropy loss

- La maximisation de L est équivalent aussi à la maximisation de la fonction log-vraisemblance :

$$LL(a) = \ln(L(a)) = \sum_{i=1}^{m} [y(i) \ln(F(x(i))) + (1 - y(i)) \ln(1 - F(x(i)))]$$

Cost function - Cross-Entropy loss

- La maximisation de L est équivalent aussi à la maximisation de la fonction log-vraisemblance :

$$LL(a) = \ln(L(a)) = \sum_{i=1}^{m} [y(i) \ln(F(x(i))) + (1 - y(i)) \ln(1 - F(x(i)))]$$

- Ce qui est équivalent à minimiser la fonction coût suivante

$$J(a) = -\frac{1}{m} LL(a) = -\frac{1}{m} \sum_{i=1}^{m} [y(i) \ln(F(x(i)) + (1 - y(i)) \ln(1 - F(x(i)))] = \frac{1}{m} \sum_{i=1}^{m} J_i(a)$$

où $J_i(a) = -[y(i) \ln(F(x(i)) + (1 - y(i)) \ln(1 - F(x(i)))]$. **Cross-Entropy loss**

Cost function - Cross-Entropy loss

- La maximisation de L est équivalent aussi à la maximisation de la fonction log-vraisemblance :

$$LL(a) = \ln(L(a)) = \sum_{i=1}^{m} [y(i) \ln(F(x(i))) + (1 - y(i)) \ln(1 - F(x(i)))]$$

- Ce qui est équivalent à minimiser la fonction coût suivante

$$J(a) = -\frac{1}{m} LL(a) = -\frac{1}{m} \sum_{i=1}^{m} [y(i) \ln(F(x(i)) + (1 - y(i)) \ln(1 - F(x(i)))] = \frac{1}{m} \sum_{i=1}^{m} J_i(a)$$

où $J_i(a) = -[y(i) \ln(F(x(i)) + (1 - y(i)) \ln(1 - F(x(i)))]$. **Cross-Entropy loss**

- Si $y(i) = 0$, alors $J_i(a) = -\ln(1 - F(x(i)))$ qui atteint son minimum si $F(x(i)) = 0$
- Si $y(i) = 1$, alors $J_i(a) = -\ln(F(x(i)))$ qui atteint son minimum si $F(x(i)) = 1$

Cost function - Cross-Entropy loss

- La maximisation de L est équivalent aussi à la maximisation de la fonction log-vraisemblance :

$$LL(a) = \ln(L(a)) = \sum_{i=1}^{m} [y(i) \ln(F(x(i))) + (1 - y(i)) \ln(1 - F(x(i)))]$$

- Ce qui est équivalent à minimiser la fonction coût suivante

$$J(a) = -\frac{1}{m} LL(a) = -\frac{1}{m} \sum_{i=1}^{m} [y(i) \ln(F(x(i)) + (1 - y(i)) \ln(1 - F(x(i)))] = \frac{1}{m} \sum_{i=1}^{m} J_i(a)$$

où $J_i(a) = -[y(i) \ln(F(x(i)) + (1 - y(i)) \ln(1 - F(x(i)))]$. **Cross-Entropy loss**

  - Si $y(i) = 0$, alors $J_i(a) = -\ln(1 - F(x(i)))$ qui atteint son minimum si $F(x(i)) = 0$
  - Si $y(i) = 1$, alors $J_i(a) = -\ln(F(x(i)))$ qui atteint son minimum si $F(x(i)) = 1$

- Le gradient de la fonction J par rapport à a est donné par

$$\nabla_a J = X^t (F(X) - Y)$$

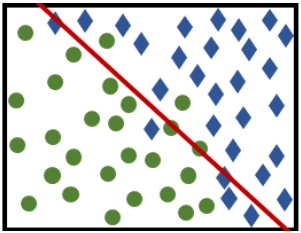
Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

Régularisation

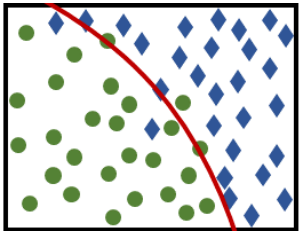
Generalization

Underfitting



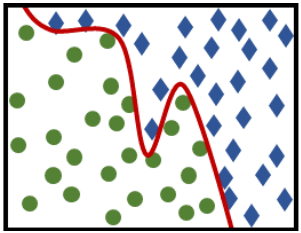
$$F(x) = \frac{1}{1+e^{-(a_0+a_1x_1+a_2x_2)}}$$

Balanced



$$F(x) = \frac{1}{1+e^{-p_2(x)}}$$

Overfitting



$$F(x) = \frac{1}{1+e^{-p_{10}(x)}}$$

Régularisation

- La régularisation du modèle permet d'éviter le problème d'overfitting en pénalisant les fortes valeurs des poids. Il consiste à utiliser une norme du vecteur des poids qu'on rajoute à la fonction à minimiser.
- Les normes les plus utiliser sont '$\ell 2$' et '$\ell 1$'. Pour la norme '$\ell 2$', la fonction coût devient

$$J(a) = -\frac{1}{m}\sum_{i=1}^{m}[y(i)\ln(F(x(i)) + (1-y(i))\ln(1-F(x(i)))] + \beta\|\tilde{a}\|_2^2$$

où β est un **paramètre de régularisation**, $\tilde{a} = (0, a_1, a_2, \cdots, a_p)$ et $\|\tilde{a}\|_2^2 = a^t a = \sum_{i=1}^{p} a_i^2$
Attention, le biais a_0 n'est pas pénalisé.

- La minimisation s'effectue par la méthode de "Gradient descent"

$$a = a - \alpha\nabla_a J$$

où $\nabla_a J = X^t(F(X) - Y) + 2\beta\tilde{a}$.

Régularisation

- Le choix du paramètre de régularisation β n'est pas évident. En effet,
  - β très grand correspond à une pénalisation forte. Ainsi, les poids seront forcés d'être très petits ce qui conduit à un problème d'underfitting (biais fort).
  - β très petit est synonyme d'une pénalisation très faible des poids ce qui conduit à un problème d'overfitting (variance forte).

- Il est alors nécessaire de choisir β de façon à assurer un compromis entre l'erreur de biais et l'erreur de variance.

- Dans la pratique, on teste différentes valeurs de β, puis on choisit celle qui réalise un bon score sur les données de test.

Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

Confusion matrix

- La matrice de confusion résume les résultats d'une classification

		Predicted		
		Positif	Négatif	
Observed	Positif	TP	FN	P
	Négatif	FP	TN	N
		PP	PN	

TP (True Positives) : les vrais positifs

TN (True Negatives) : les vrais négatifs

FP (False Positive) : les faux positifs

FN (False Negative) : les faux négatifs

P=TP+FN : les positifs

N=TN+FP : les négatifs

PP=TP+FP : les prédits positifs

PN=TN+FP : les prédits négatifs

Confusion matrix

- **Accuracy** $= \frac{TP+TN}{P+N}$
  - Rapport des bonnes prédictions. Une valeur proche de 1 correspond à une bonne accuracy, tandis qu'une valeur proche de 0 est synonyme d'une mauvaise accuracy.
  - Indicateur très simple à comprendre. Mais le coût d'erreur sur les positifs et les négatifs est souvent différent.
  - N'est pas fiable si les classes sont déséquilibrées

Confusion matrix

- **Accuracy** $= \frac{TP+TN}{P+N}$
  - Rapport des bonnes prédictions. Une valeur proche de 1 correspond à une bonne accuracy, tandis qu'une valeur proche de 0 est synonyme d'une mauvaise accuracy.
  - Indicateur très simple à comprendre. Mais le coût d'erreur sur les positifs et les négatifs est souvent différent.
  - N'est pas fiable si les classes sont déséquilibrées
- **Precision** $= \frac{TP}{PP}$
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des prédits positifs (PP).
  - Une valeur de la précision proche de 1 implique une bonne précision ce qui indique que les faux positifs sans rare.

Confusion matrix

- **Accuracy** $= \frac{TP+TN}{P+N}$
  - Rapport des bonnes prédictions. Une valeur proche de 1 correspond à une bonne accuracy, tandis qu'une valeur proche de 0 est synonyme d'une mauvaise accuracy.
  - Indicateur très simple à comprendre. Mais le coût d'erreur sur les positifs et les négatifs est souvent différent.
  - N'est pas fiable si les classes sont déséquilibrées
- **Precision** $= \frac{TP}{PP}$
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des prédits positifs (PP).
  - Une valeur de la précision proche de 1 implique une bonne précision ce qui indique que les faux positifs sans rare.
- **Sensibility** $= \frac{TP}{P}$ (Recall, true positive rate: Tpr)
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des positifs (P).
  - La valeur 1 correspond à une meilleure sensibilité, ce qui implique que le modèle arrive à détecter tous les positifs.

Confusion matrix

- **Accuracy** $= \frac{TP+TN}{P+N}$
  - Rapport des bonnes prédictions. Une valeur proche de 1 correspond à une bonne accuracy, tandis qu'une valeur proche de 0 est synonyme d'une mauvaise accuracy.
  - Indicateur très simple à comprendre. Mais le coût d'erreur sur les positifs et les négatifs est souvent différent.
  - N'est pas fiable si les classes sont déséquilibrées
- **Precision** $= \frac{TP}{PP}$
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des prédits positifs (PP).
  - Une valeur de la précision proche de 1 implique une bonne précision ce qui indique que les faux positifs sans rare.
- **Sensibility** $= \frac{TP}{P}$ (Recall, true positive rate: Tpr)
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des positifs (P).
  - La valeur 1 correspond à une meilleure sensibilité, ce qui implique que le modèle arrive à détecter tous les positifs.
- **Specificity** $= \frac{TN}{N}$ (true negative rate)
  - Rapport entre le nombre de vrais négatifs (TN) et le nombre total des négatifs (N).
  - Une meilleur spécificité est 1, tandis qu'une mauvaise est 0.

Confusion matrix

- **Accuracy** $= \frac{TP+TN}{P+N}$
  - Rapport des bonnes prédictions. Une valeur proche de 1 correspond à une bonne accuracy, tandis qu'une valeur proche de 0 est synonyme d'une mauvaise accuracy.
  - Indicateur très simple à comprendre. Mais le coût d'erreur sur les positifs et les négatifs est souvent différent.
  - N'est pas fiable si les classes sont déséquilibrées
- **Precision** $= \frac{TP}{PP}$
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des prédits positifs (PP).
  - Une valeur de la précision proche de 1 implique une bonne précision ce qui indique que les faux positifs sans rare.
- **Sensibility** $= \frac{TP}{P}$ (Recall, true positive rate: Tpr)
  - Rapport entre le nombre de vrais positifs (TP) et le nombre total des positifs (P).
  - La valeur 1 correspond à une meilleure sensibilité, ce qui implique que le modèle arrive à détecter tous les positifs.
- **Specificity** $= \frac{TN}{N}$ (true negative rate)
  - Rapport entre le nombre de vrais négatifs (TN) et le nombre total des négatifs (N).
  - Une meilleur spécificité est 1, tandis qu'une mauvaise est 0.
- **Fpr** = 1- **Specificity** (false positive rate)
  - La valeur 0 correspond à un meilleur Fpr, tandis que la valeur 1 correspond à un mauvais Fpr.

Qualité d'une classification

Exemple

	^+	^-
+	90	0
-	10	0

	^+	^-
+	0	90
-	0	10

	^+	^-
+	45	45
-	0	10

	^+	^-
+	90	0
-	5	5

Accuracy = ?

Precision = ?

Sensibility = ?

Specificity = ?

Qualité d'une classification

Exemple

	^+	^-
+	90	0
-	10	0

	^+	^-
+	0	90
-	0	10

	^+	^-
+	45	45
-	0	10

	^+	^-
+	90	0
-	5	5

Accuracy = ? 0.90 0.10 0.55 0.95

Precision = ?

Sensibility = ?

Specificity = ?

Qualité d'une classification

Exemple

	^+	^-
+	90	0
-	10	0

	^+	^-
+	0	90
-	0	10

	^+	^-
+	45	45
-	0	10

	^+	^-
+	90	0
-	5	5

Accuracy = ?	0.90	0.10	0.55	0.95
Precision = ?	0.90	*nan*	1.00	0.95
Sensibility = ?				
Specificity = ?				

Qualité d'une classification

Exemple

	^+	^-
+	90	0
-	10	0

	^+	^-
+	0	90
-	0	10

	^+	^-
+	45	45
-	0	10

	^+	^-
+	90	0
-	5	5

Accuracy = ?	0.90	0.10	0.55	0.95
Precision = ?	0.90	*nan*	1.00	0.95
Sensibility = ?	1.00	0.00	0.50	1.00
Specificity = ?				

Qualité d'une classification

Exemple

	^+	^-
+	90	0
-	10	0

	^+	^-
+	0	90
-	0	10

	^+	^-
+	45	45
-	0	10

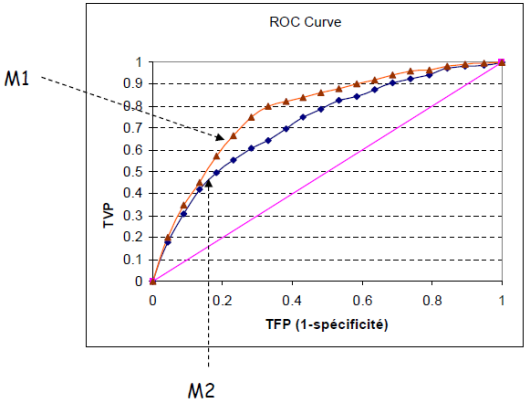
	^+	^-
+	90	0
-	5	5

Accuracy = ?	0.90	0.10	0.55	0.95
Precision = ?	0.90	*nan*	1.00	0.95
Sensibility = ?	1.00	0.00	0.50	1.00
Specificity = ?	0.00	1.00	1.00	0.50

Qualité d'une classification

Courbe ROC

- La courbe ROC est un outil graphique d'évaluation et de comparaison des modèles de classification binaire



Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+		
2	0.95	+		
3	0.9	+		
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+		
3	0.9	+		
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

$Tpr = 1/6 = 0.167$ et $Fpr = 0/14 = 0$

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+		
3	0.9	+		
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+		
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

$Tpr = 2/6 = 0.333$ et $Fpr = 0/14 = 0$

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+		
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

	^positif	^négatif	Total
positf	3	3	6
négatif	0	14	14
Total	3	17	20

$Tpr = 3/6 = 0.5$ et $Fpr = 0/14 = 0$

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-		
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

	^positif	^négatif	Total
positf	3	3	6
négatif	0	14	14
Total	3	17	20

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-	0.500	0.071
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

	^positif	^négatif	Total
positf	3	3	6
négatif	0	14	14
Total	3	17	20

	^positif	^négatif	Total
positf	3	3	6
négatif	1	13	14
Total	4	16	20

$Tpr = 3/6 = 0.5$ et $Fpr = 1/14 = 0.071$

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-	0.500	0.071
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-		

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

	^positif	^négatif	Total
positf	3	3	6
négatif	0	14	14
Total	3	17	20

	^positif	^négatif	Total
positf	3	3	6
négatif	1	13	14
Total	4	16	20

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-	0.500	0.071
5	0.8	+		
6	0.75	-		
7	0.7	-		
8	0.65	+		
9	0.6	-		
10	0.55	-		
11	0.5	-		
12	0.45	+		
13	0.4	-		
14	0.35	-		
15	0.3	-		
16	0.25	-		
17	0.2	-		
18	0.15	-		
19	0.1	-		
20	0.05	-	1.000	1.000

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

	^positif	^négatif	Total
positf	3	3	6
négatif	0	14	14
Total	3	17	20

	^positif	^négatif	Total
positf	3	3	6
négatif	1	13	14
Total	4	16	20

$Tpr = 6/6 = 1$ et $Fpr = 14/14 = 1$

	^positif	^négatif	Total
positf	6	0	6
négatif	14	0	14
Total	20	0	20

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-	0.500	0.071
5	0.8	+	0.667	0.071
6	0.75	-	0.667	0.143
7	0.7	-	0.667	0.214
8	0.65	+	0.833	0.214
9	0.6	-	0.833	0.286
10	0.55	-	0.833	0.357
11	0.5	-	0.833	0.429
12	0.45	+	1.000	0.429
13	0.4	-	1.000	0.500
14	0.35	-	1.000	0.572
15	0.3	-	1.000	0.643
16	0.25	-	1.000	0.714
17	0.2	-	1.000	0.786
18	0.15	-	1.000	0.857
19	0.1	-	1.000	0.929
20	0.05	-	1.000	1.000

	^positif	^négatif	Total
positf	1	5	6
négatif	0	14	14
Total	1	19	20

	^positif	^négatif	Total
positf	2	4	6
négatif	0	14	14
Total	2	18	20

	^positif	^négatif	Total
positf	3	3	6
négatif	0	14	14
Total	3	17	20

	^positif	^négatif	Total
positf	3	3	6
négatif	1	13	14
Total	4	16	20

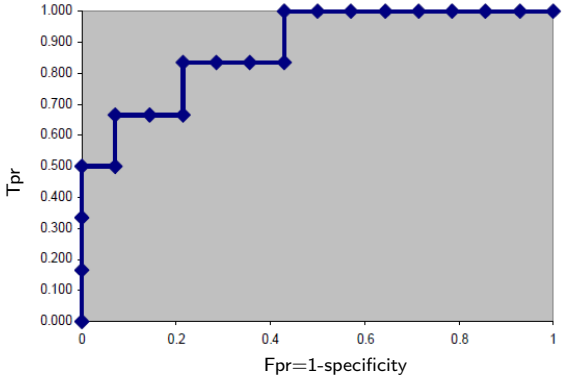
	^positif	^négatif	Total
positf	6	0	6
négatif	14	0	14
Total	20	0	20

Qualité d'une classification

Courbe ROC

Classer les données
selon un score décroissant

Individu	Score (+)	Classe	Tpr	Fpr
1	1	+	0.167	0.000
2	0.95	+	0.333	0.000
3	0.9	+	0.500	0.000
4	0.85	-	0.500	0.071
5	0.8	+	0.667	0.071
6	0.75	-	0.667	0.143
7	0.7	-	0.667	0.214
8	0.65	+	0.833	0.214
9	0.6	-	0.833	0.286
10	0.55	-	0.833	0.357
11	0.5	-	0.833	0.429
12	0.45	+	1.000	0.429
13	0.4	-	1.000	0.500
14	0.35	-	1.000	0.572
15	0.3	-	1.000	0.643
16	0.25	-	1.000	0.714
17	0.2	-	1.000	0.786
18	0.15	-	1.000	0.857
19	0.1	-	1.000	0.929
20	0.05	-	1.000	1.000



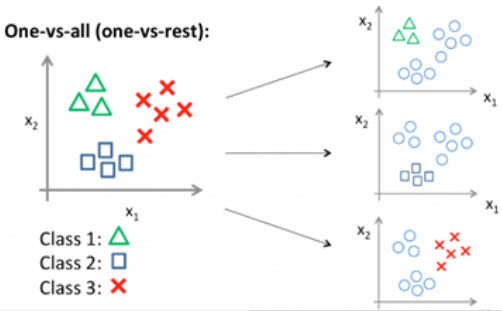
Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

One-vs-all (one-vs-rest)

- On parle de Multi-class classification lorsqu'on a K catégories avec $K > 2$ ($y \in \{0, 1, 2, \cdots, K-1\}$).
- On découpe le problème en K problèmes de classification binaires.
- L'une des trois classes est considérée la classe positive et le reste comme la classe négative.
- Après entrainement, on obtient K fonctions de prédiction $F^i(x),\ i = 1, \cdots, K$.
- La classe de l'observation x est

$$\arg \max_{i \in \{1, \cdots, K\}} F^i(x)$$



Softmax Regression

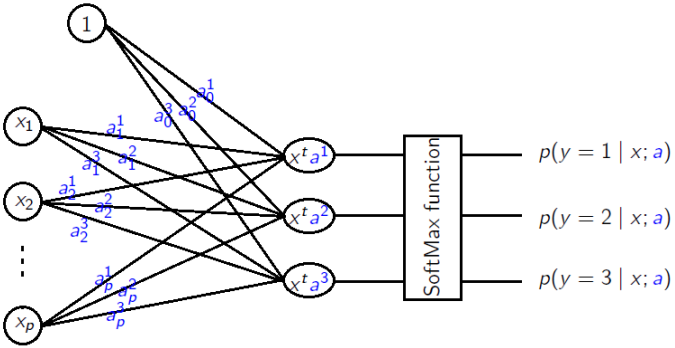
Likelihood

- La fonction Softmax prend en entrée un vecteur $\mathbf{z} = (z_1, \cdots, z_K)$ et renvoie un vecteur $S(\mathbf{z})$ de K nombres réels strictement positifs et de somme 1

$$S(\mathbf{z}) = \frac{1}{\sum_{k \in \{1, \cdots, K\}} e^{z_k}} \begin{bmatrix} e^{z_1} \\ \vdots \\ e^{z_K} \end{bmatrix}$$

- En classification Multi-class, la fonction Softmax renvoie un vecteur des probabilités d'appartenance à chaque classe.

Softmax Regression



$$F(x) = \frac{1}{\sum_{k \in \{1,2,3\}} e^{x^t a(k)}} \begin{bmatrix} e^{x^t a^1} \\ e^{x^t a^2} \\ e^{x^t a^3} \end{bmatrix}$$

Softmax Regression

- On note le vecteur des K classes $\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_K \end{bmatrix}$ avec un encodage **one-hot**.
- Pour une observation, on cherche alors à maximiser la vraisemblance (likelihood) suivante :

$$p(\mathbf{y}/x) = \prod_{k=1}^{K} p(y_k/x)^{y_k} = \prod_{k=1}^{K} (F(x))_k^{y_k}$$

- Ceci revient à minimiser la "negative log-likelihood function"

$$-\log\left(\prod_{k=1}^{K} (F(x))_k^{y_k}\right) = -\sum_{k=1}^{K} y_k \log(F(x)_k)$$

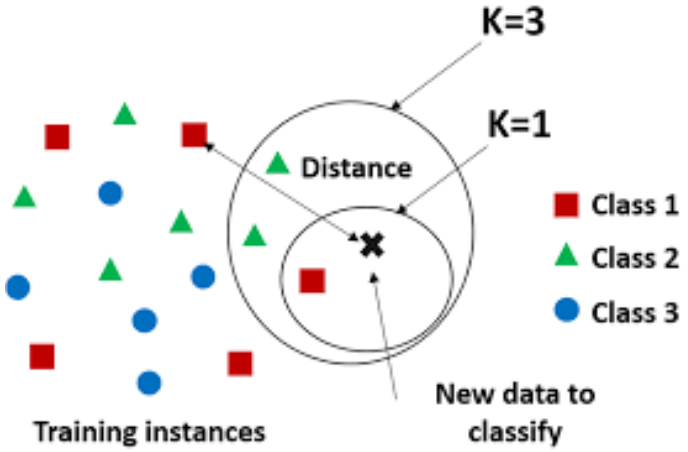
- The cross-entropy error function over a training set of size m can be calculated as:

$$J(a) = -\sum_{i=1}^{m}\sum_{k=1}^{K} y_k(i) \log(F(x(i))_k)$$

Sommaire

1. Régression logistique
2. Cost function
3. Régularisation
4. Qualité d'une classification
5. Multiclasse
6. K-nearest neighbors

K-nearest neighbors (KNN)



Principe de K-NN : dis moi qui sont tes voisins, je te dirais qui tu es!

K-nearest neighbors (KNN)

- Données en entrée :
  - un ensemble de données D,
  - une distance d (euclidienne, Manhattan, ...),
  - un nombre entier K.
- Pour une nouvelle observation X dont on veut prédire sa variable de sortie y Faire :
  1. Calculer toutes les distances entre X et les autres observations de D
  2. Retenir les K observations les proches de X
  3. Prendre les valeurs de y des K observations retenues :
     - Si on effectue une régression, calculer la moyenne des y retenues
     - Si on effectue une classification , calculer le mode des y retenues
  4. Retourner la valeur calculée dans l'étape 3 comme étant la valeur qui a été prédite par K-NN pour l'observation X

End Lecture N° 2

Machine Learning

Réseaux de neurones artificiels

Mohamed Berrada

Ecole Nationale Supérieure des Arts et Métiers de Meknès
Université Moulay Ismail
Filière GI-IADS

2021-2022

Sommaire

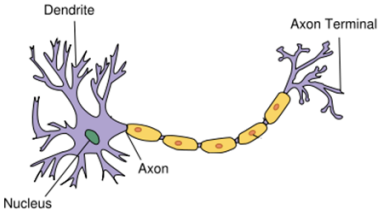
1. Perceptron
2. Multilayer perceptron MLP
3. Backpropagation
4. Validation croisée
5. Tuning the hyper-parameters

Sommaire

1. Perceptron
2. Multilayer perceptron MLP
3. Backpropagation
4. Validation croisée
5. Tuning the hyper-parameters

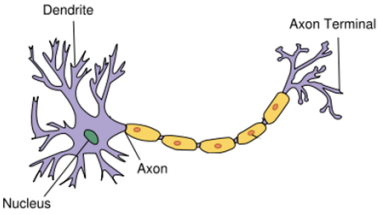
Perceptron

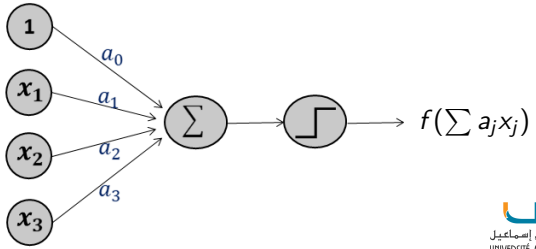
- Le neurone biologique est composé de dendrites, d'un corps cellulaire et d'axone.



Perceptron

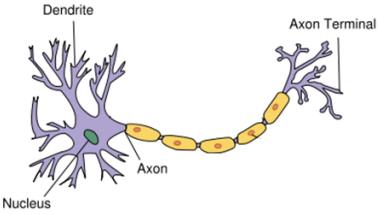
- Le neurone biologique est composé de dendrites, d'un corps cellulaire et d'axone.
- Le perceptron est une entité de calcul élémentaire inspirée du neurone biologique

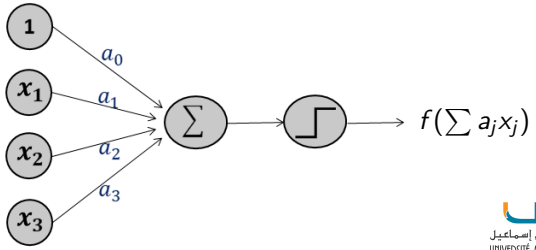




Perceptron

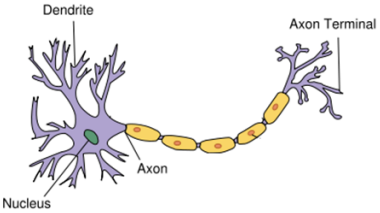
- Le neurone biologique est composé de dendrites, d'un corps cellulaire et d'axone.
- Le perceptron est une entité de calcul élémentaire inspirée du neurone biologique
- Chaque connexion entre le neurone et ses entrées est dotée d'un poids a_j.

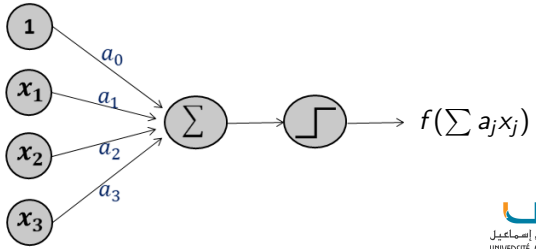




Perceptron

- Le neurone biologique est composé de dendrites, d'un corps cellulaire et d'axone.
- Le perceptron est une entité de calcul élémentaire inspirée du neurone biologique
- Chaque connexion entre le neurone et ses entrées est dotée d'un poids a_j.
- Le neurone fait la somme pondérée de ses entrées, puis passe cette somme à travers une fonction d'activation.
  Plusieurs choix de fonction d'activation sont possibles (linéaire, sigmoïde, tanh, reLu ...).

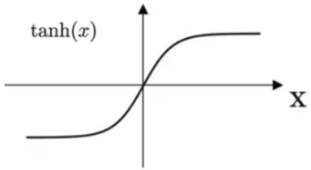




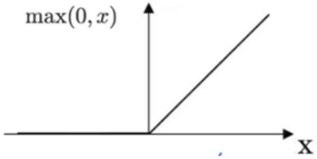
Perceptron

Fonctions d'activation :

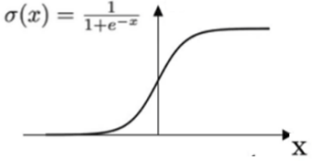
Hyper Tangent Function



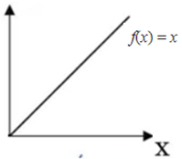
ReLU Function



Sigmoid Function



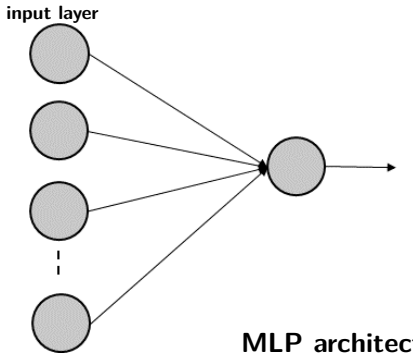
Identity Function



Sommaire

1. Perceptron
2. Multilayer perceptron MLP
3. Backpropagation
4. Validation croisée
5. Tuning the hyper-parameters

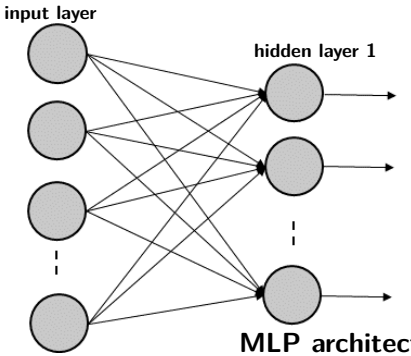
Multilayer perceptron MLP



MLP architecture (Feed Forward)

- Chacun des neurones d'une couche est connecté à tous les neurones de la couche suivante.
- Chaque connexion a un poids a_{ij}.
- L'information se propage alors de couche en couche sans retour en arrière possible.
- Pour chaque couche du réseau, il y a un terme de biais.

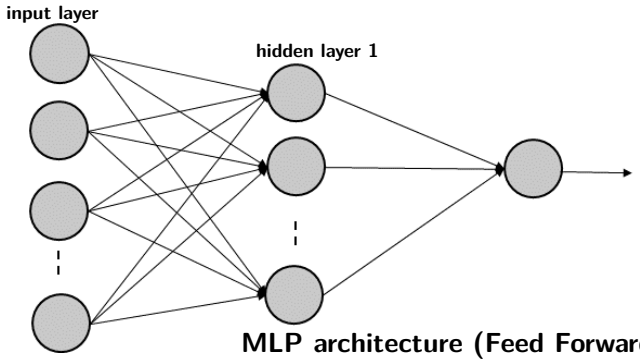
Multilayer perceptron MLP



MLP architecture (Feed Forward)

- Chacun des neurones d'une couche est connecté à tous les neurones de la couche suivante.
- Chaque connexion a un poids a_{ij}.
- L'information se propage alors de couche en couche sans retour en arrière possible.
- Pour chaque couche du réseau, il y a un terme de biais.

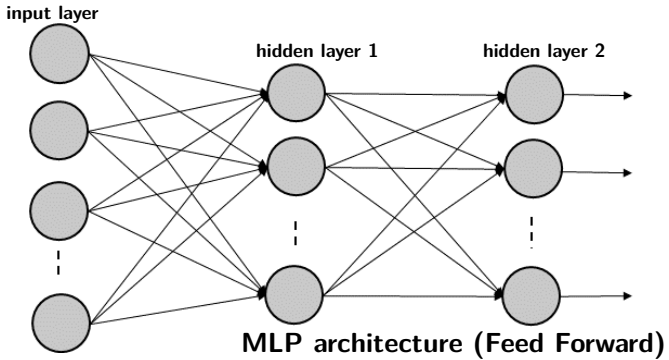
Multilayer perceptron MLP



MLP architecture (Feed Forward)

- Chacun des neurones d'une couche est connecté à tous les neurones de la couche suivante.
- Chaque connexion a un poids a_{ij}.
- L'information se propage alors de couche en couche sans retour en arrière possible.
- Pour chaque couche du réseau, il y a un terme de biais.

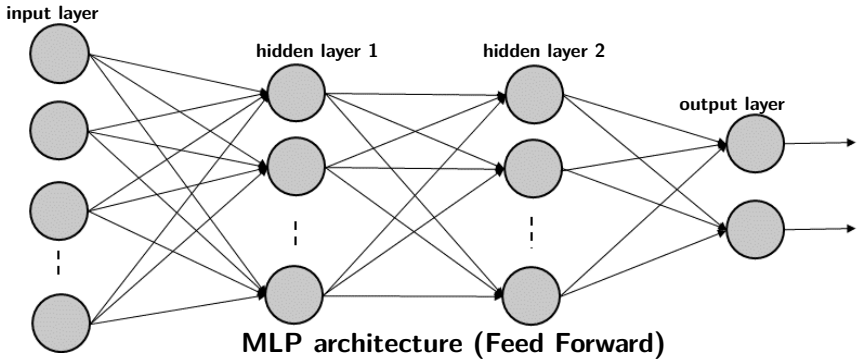
Multilayer perceptron MLP



MLP architecture (Feed Forward)

- Chacun des neurones d'une couche est connecté à tous les neurones de la couche suivante.
- Chaque connexion a un poids a_{ij}.
- L'information se propage alors de couche en couche sans retour en arrière possible.
- Pour chaque couche du réseau, il y a un terme de biais.

Multilayer perceptron MLP



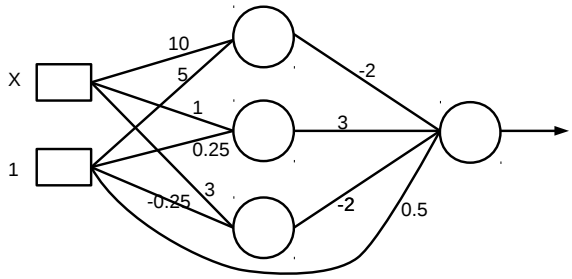
MLP architecture (Feed Forward)

- Chacun des neurones d'une couche est connecté à tous les neurones de la couche suivante.
- Chaque connexion a un poids a_{ij}.
- L'information se propage alors de couche en couche sans retour en arrière possible.
- Pour chaque couche du réseau, il y a un terme de biais.

Multilayer perceptron MLP

Exercice :

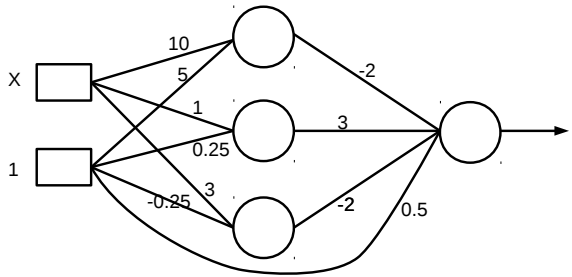
Sachant que la fonction d'activation de la couche cachée est le tangent hyperbolique et celle de la couche de sortie est l'identité. Quelle est la sortie de ce réseau de neurones?



Multilayer perceptron MLP

Exercice :

Sachant que la fonction d'activation de la couche cachée est le tangent hyperbolique et celle de la couche de sortie est l'identité. Quelle est la sortie de ce réseau de neurones?



$$G = -2th(10x + 5) + 3th(x + 0.25) - 2th(3x - 0.25) + 0.5$$

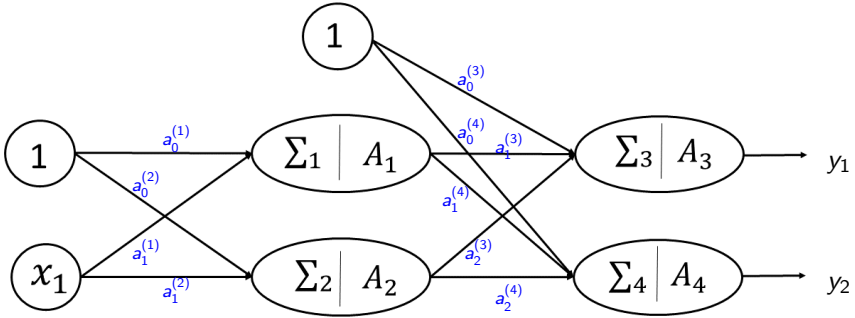
Multilayer perceptron MLP

- Pour la classification en utilise la fonction `MLPClassifier` du module `sklearn.neural_network`
  - Output activation : softmax (multiclass) or logistic (binary)
  - Fonction coût : the Cross-Entropy loss function
- Pour la classification en utilise la fonction `MLPRegressor` du module `sklearn.neural_network`
  - Output activation : identity
  - Fonction coût : the Squared-error loss function

Sommaire

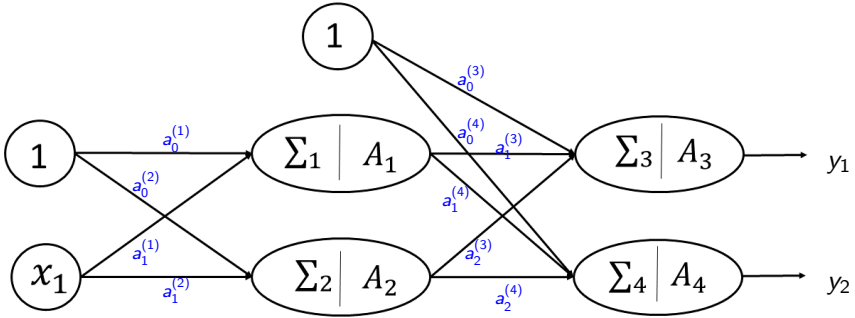
1. Perceptron
2. Multilayer perceptron MLP
3. Backpropagation
4. Validation croisée
5. Tuning the hyper-parameters

Backpropagation



Activation: $A_1 = A_2 = \tanh$ and $A_3 = A_4 = \textit{Identity}$

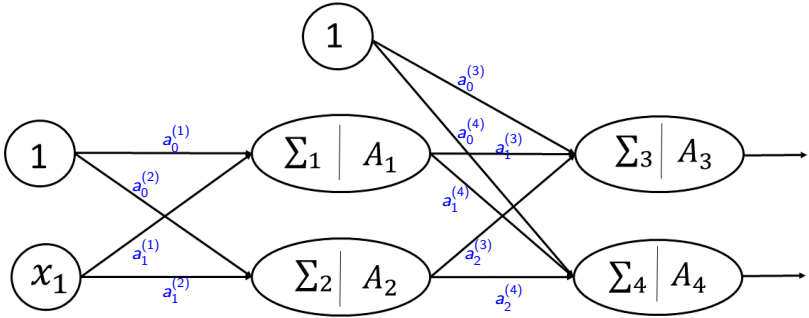
Backpropagation



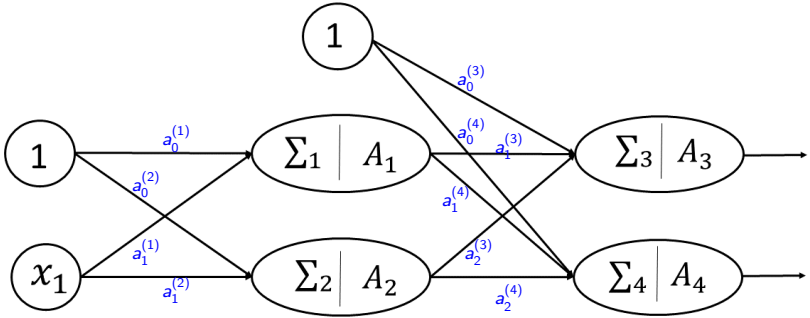
Activation: $A_1 = A_2 = \tanh$ and $A_3 = A_4 = \textit{Identity}$

$$y_1 = a_0^{(3)} + a_1^{(3)} \tanh(a_0^{(1)} + a_1^{(1)} x) + a_2^{(3)} \tanh(a_0^{(2)} + a_1^{(2)} x)$$
$$y_2 = a_0^{(4)} + a_1^{(4)} \tanh(a_0^{(1)} + a_1^{(1)} x) + a_2^{(4)} \tanh(a_0^{(2)} + a_1^{(2)} x)$$

Backpropagation

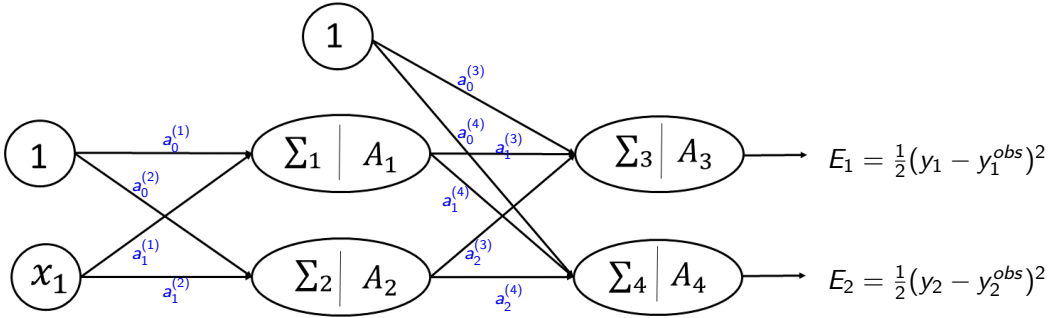


Backpropagation



For an observation

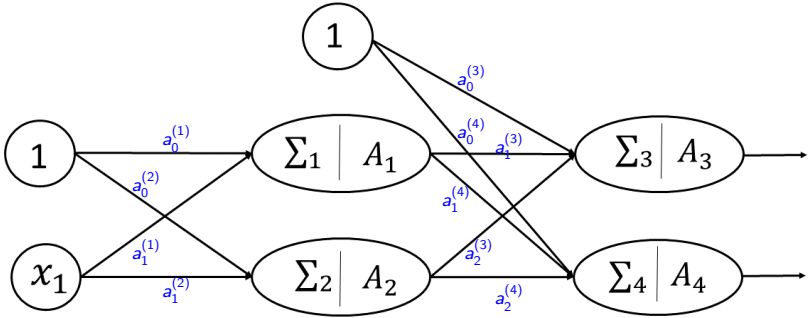
Backpropagation



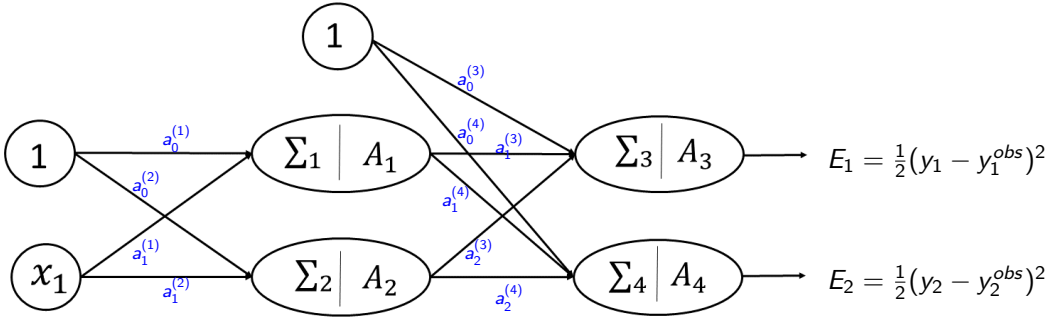
For an observation

$$J(a) = E_1 + E_2 = \frac{1}{2}\left((y_1 - y_1^{obs})^2 + (y_2 - y_2^{obs})^2\right)$$

Backpropagation

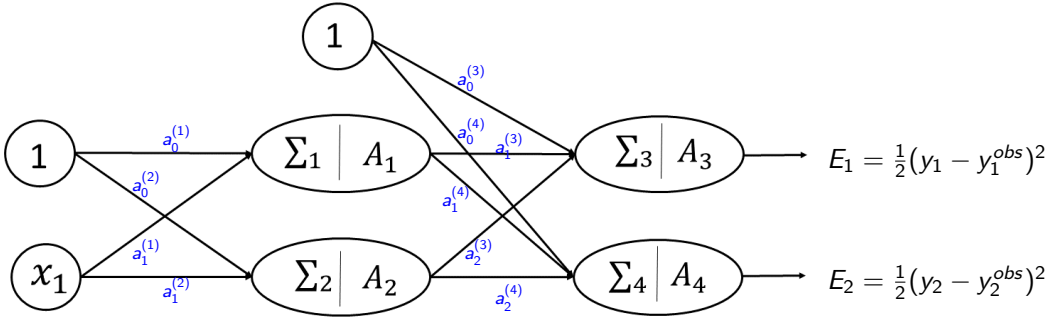


Backpropagation



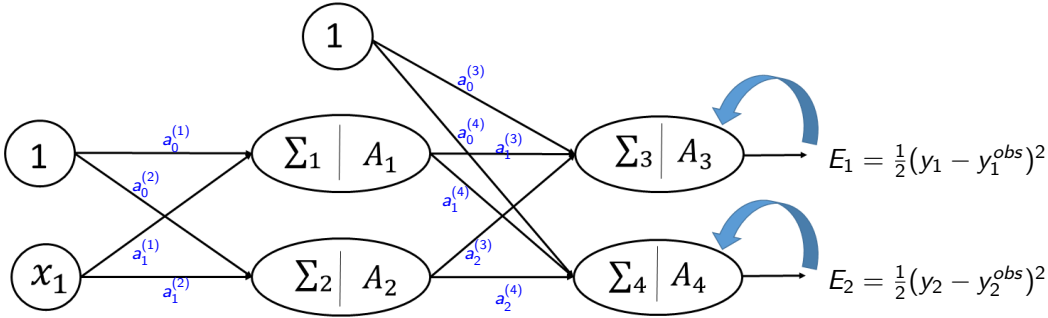
Gradient of J in terms of $a_0^{(3)}$: $\frac{\partial J}{\partial a_0^{(3)}} = \frac{\partial E_1}{\partial A_3} \frac{\partial A_3}{\partial \Sigma_3} \frac{\partial \Sigma_3}{\partial a_0^{(3)}}$

Backpropagation



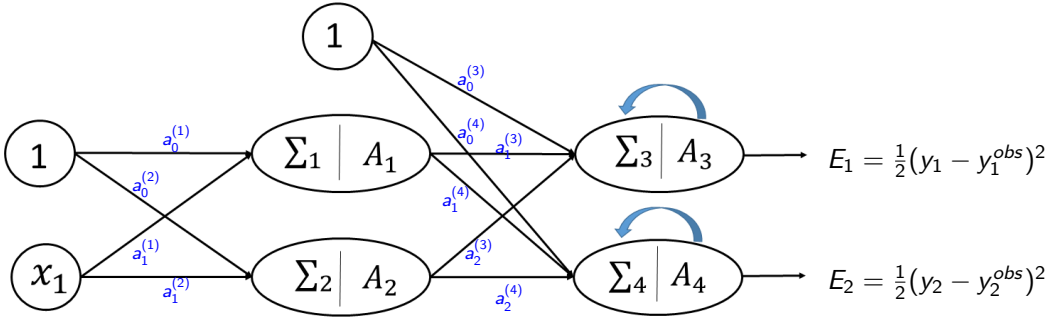
Gradient of J in terms of $a_0^{(3)}$: $\frac{\partial J}{\partial a_0^{(3)}} = \frac{\partial E_1}{\partial A_3} \frac{\partial A_3}{\partial \Sigma_3} \frac{\partial \Sigma_3}{\partial a_0^{(3)}}$

Backpropagation



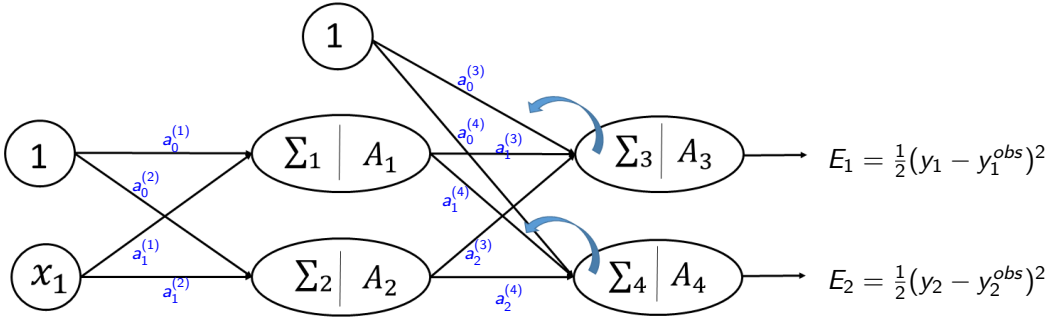
Gradient of J in terms of $a_0^{(3)}$: $\frac{\partial J}{\partial a_0^{(3)}} = \frac{\partial E_1}{\partial A_3} \frac{\partial A_3}{\partial \Sigma_3} \frac{\partial \Sigma_3}{\partial a_0^{(3)}}$

Backpropagation



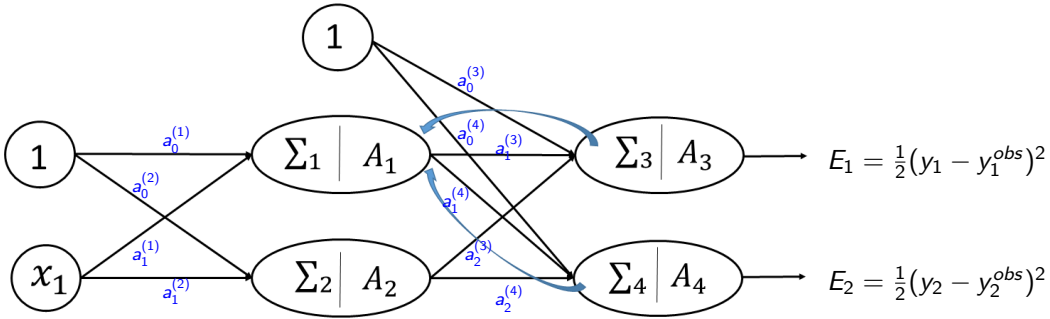
Gradient of J in terms of $a_0^{(3)}$: $\frac{\partial J}{\partial a_0^{(3)}} = \frac{\partial E_1}{\partial A_3} \frac{\partial A_3}{\partial \Sigma_3} \frac{\partial \Sigma_3}{\partial a_0^{(3)}}$

Backpropagation



Gradient of J in terms of $a_0^{(3)}$: $\frac{\partial J}{\partial a_0^{(3)}} = \frac{\partial E_1}{\partial A_3} \frac{\partial A_3}{\partial \Sigma_3} \frac{\partial \Sigma_3}{\partial a_0^{(3)}}$

Backpropagation



Gradient of J in terms of $a_0^{(3)}$: $\frac{\partial J}{\partial a_0^{(3)}} = \frac{\partial E_1}{\partial A_3} \frac{\partial A_3}{\partial \Sigma_3} \frac{\partial \Sigma_3}{\partial a_0^{(3)}}$

- What is the gradient of J in terms of A_1?
- What is the gradient of A_1 in terms of $a_0^{(1)}$?

Sommaire

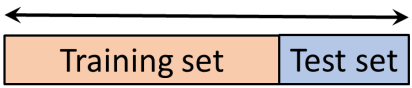
1. Perceptron
2. Multilayer perceptron MLP
3. Backpropagation
4. Validation croisée
5. Tuning the hyper-parameters

Validation croisée

Validation croisée

La validation croisée est une procédure de rééchantillonnage permettant d'évaluer un modèle d'apprentissage automatique et de tester sa capacité de faire des prédictions sur des données de test indépendant. Elle permet ainsi de choisir lequel entre deux modèles représente mieux le vrai modèle de nos données.

La performance dépend du "data splitting"



L'évaluation doit être faite au cours du développement de modèle indépendamment du test.

Validation croisée

Leave-P-Out Cross Validation

On utilise des échantillons de taille p pour la validation et les n-p échantillons qui restent comme ensemble d'apprentissage. On considère toutes les combinaisons possibles. Le nombre d'itérations est alors C_n^p.

Leave-One-Out



Validation croisée

Leave-P-Out Cross Validation

On utilise des échantillons de taille p pour la validation et les n-p échantillons qui restent comme ensemble d'apprentissage. On considère toutes les combinaisons possibles. Le nombre d'itérations est alors C_n^p.

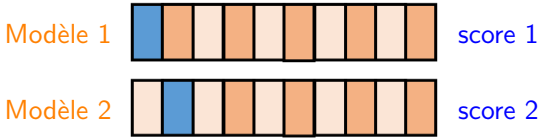
Leave-One-Out Modèle 1  score 1

Validation croisée

Leave-P-Out Cross Validation

On utilise des échantillons de taille p pour la validation et les n-p échantillons qui restent comme ensemble d'apprentissage. On considère toutes les combinaisons possibles. Le nombre d'itérations est alors C_n^p.

Leave-One-Out

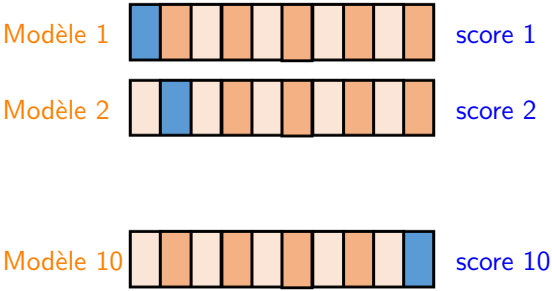


Validation croisée

Leave-P-Out Cross Validation

On utilise des échantillons de taille p pour la validation et les n-p échantillons qui restent comme ensemble d'apprentissage. On considère toutes les combinaisons possibles. Le nombre d'itérations est alors C_n^p.

Leave-One-Out

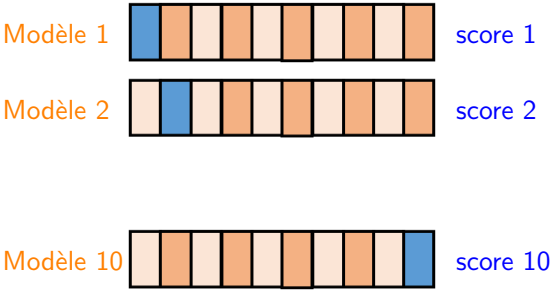


Validation croisée

Leave-P-Out Cross Validation

On utilise des échantillons de taille p pour la validation et les n-p échantillons qui restent comme ensemble d'apprentissage. On considère toutes les combinaisons possibles. Le nombre d'itérations est alors C_n^p.

Leave-One-Out

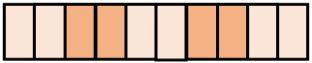


Cette méthode est couteuse lorsqu'il s'agit d'un nombre très grand d'échatillons.

Validation croisée

K-Fold Cross Validation

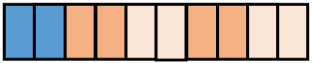
Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).



Validation croisée

K-Fold Cross Validation

Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).

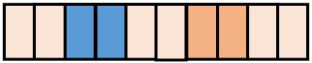


{score1, }

Validation croisée

K-Fold Cross Validation

Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).



{score1, score2, }

Validation croisée

K-Fold Cross Validation

Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).

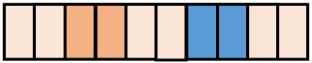


{score1, score2, score3, }

Validation croisée

K-Fold Cross Validation

Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).

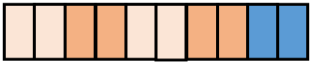


{score1, score2, score3, score4, }

Validation croisée

K-Fold Cross Validation

Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).

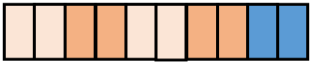


{score1, score2, score3, score4, score5}

Validation croisée

K-Fold Cross Validation

Les données sont divisées en K sous-ensembles. Pour chaque itération, un sous-ensemble est utilisé pour la validation et les autres pour l'entrainement. Le nombre d'itérations est alors égale à K (souvent on choisit K=5 à 10).



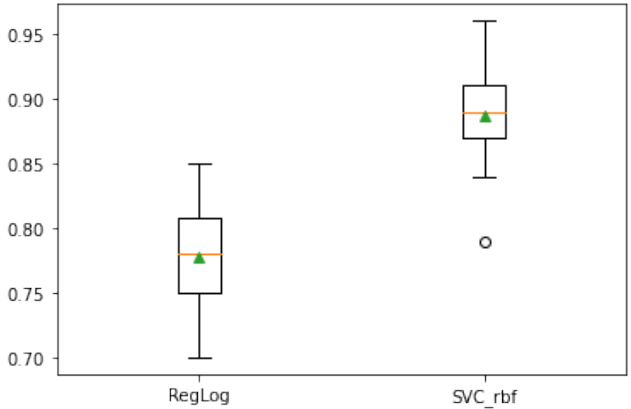
{score1, score2, score3, score4, score5}

Stratified K-Fold Cross Validation

Cette méthode de validation croisée est une légère variation de K-Fold, de sorte que chaque 'fold' contient approximativement le même pourcentage d'échantillons de chaque classe cible, ou en cas de problèmes de prédiction, la valeur de réponse moyenne est approximativement égale dans tous les 'folds'.

Validation croisée

Exemple de comparaison entre deux modèles :



Sommaire

1. Perceptron
2. Multilayer perceptron MLP
3. Backpropagation
4. Validation croisée
5. Tuning the hyper-parameters

Tuning the hyper-parameters

Les hyper-paramètres sont des paramètres qui ne sont pas directement appris dans le modèle. Les exemples typiques incluent le paramètre de régularisation, l'architecture du réseau, le noyau,...

La commande suivante permet de récupérer les paramètres d'un modèle (de la librairie sklearn): `model.get_params()`

La fonction `GridSearchCV` de `sklearn.model_selection` considère de manière exhaustive toutes les combinaisons de paramètres de la grille.

Tuning the hyper-parameters

Voici des exemples de grilles :

```
grid={"C":[0.0001, 0.001, 0.01, 0.1, 1.0], "penalty":["l2","l1"]}

grid=[{"penalty":["none"]}, {"C":[0.0001, 0.001, 0.01, 0.1, 1.0], "penalty":["l2","l1"]}]

grid={
'learning_rate':  ["constant", "invscaling", "adaptive"],
'hidden_layer_sizes':  [(100,1), (100,2), (100,3)],
'alpha':  [10.0 ** -np.arange(1, 7)],
'activation':  ["logistic", "relu", "Tanh"] }

grid={'C': [0.1,1, 10, 100], 'gamma':  [1,0.1,0.01,0.001],'kernel':  ['rbf', 'poly', 'sigmoid']}
```

Utilisation :

```
model_cv=GridSearchCV(model,grid,cv=10)
model_cv.fit(X,y)

print("tuned hpyerparameters :(best parameters) ",model_cv.best_params_)
print("accuracy :",model_cv.best_score_)
```

End Lecture N° 3

Machine learning

Support Vector Machine (SVM)

M. Berrada

Ecole Nationale Supérieure des Arts et Métiers de Meknès
Université Moulay Ismail

March 2022

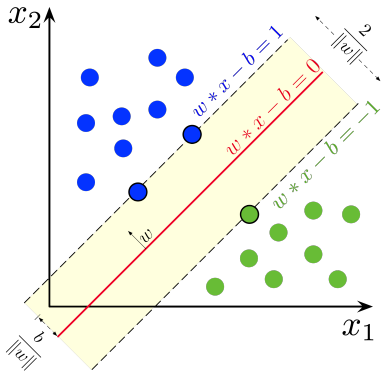
Sommaire

1. Introduction
2. Problème primal
3. Multiplicateurs de Lagrange
4. Problème dual
5. Marge souple (soft margin)
6. SVM à Noyau
7. Cas multi-classes
8. SVM pour la régression

Introduction

Ensemble d'apprentissage $\{(x_i, y_i),\ i = 1, \cdots, M\}$ tel que $x_i \in \mathbb{R}^N$ et $y \in \{-1, +1\}$.

- Construire une fonction f telle que
  - Si $f(x) > 0$, x est dans la classe +1
  - Si $f(x) < 0$, x est dans la classe -1
  - Si $f(x) = 0$, x est dans l'hyperplan de séparation
- Cas linéaire
  $$f(x) = w \cdot x + b \text{ tel que } w \in \mathbb{R}^N \text{ et } b \in \mathbb{R}.$$
- Hyperplan de séparation : $\mathcal{H} = \{x \mid w \cdot x + b = 0\}$



Quel est le meilleur hyperplan $\mathcal{H}$?
Hyperplan de marge maximale (séparateur à vaste marge)

$$Marge(\mathcal{H}) = \min_{x_i} d(x_i, \mathcal{H})$$

Vecteurs supports

Les vecteurs supports sont les éléments les plus proches de l'hyperplan. Ils sont notés x_s.

- Leur distance à l'hyperplan est égale à la marge

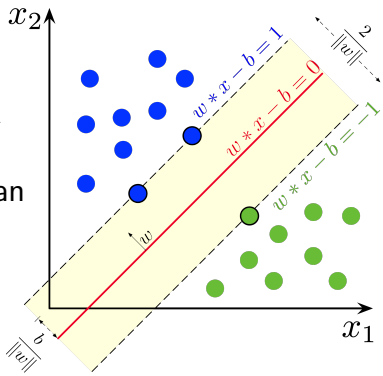
$$d(x_s, \mathcal{H}) = \frac{|w.x_s + b|}{\|w\|}$$

- Les paramètres kw et kb donnent le même hyperplan

$$kw.x + kb = k(w.x + b) = 0$$

- Condition de normalisation :
  On choisit w et b tel que $|w.x_s + b| = 1$
  - Si $y_i = +1$, alors $w.x_i + b \geq 1$
  - Si $y_i = -1$, alors $w.x_i + b \leq -1$
- La marge qu'on doit maximiser

$$d(x_s, \mathcal{H}) = \frac{1}{\|w\|}$$



Problème primal

Objectif

Maximisation la marge $\frac{1}{\|w\|}$ sous les contraintes $y_i(w.x_i + b) \geq 1,\ i = 1, \cdots, M.$

Problème équivalent

Le problème de la maximisation de la marge est équivalent au problème de minimisation, dit primal, suivant:

$$\text{Minimiser} \quad \frac{1}{2}\|w\|^2, \text{ sous les contraintes} \quad y_i(w.x_i + b) \geq 1,\ i = 1, \cdots, M$$

- Problème d'optimisation convexe sous contraintes linéaires
- Existence d'un optimum global, obtenu par résolution du problème dual

Multiplicateurs de Lagrange

Soit $h : \mathbb{R}^d \to \mathbb{R}$

Problème : Minimiser $h(u)$ sous contraintes $g_i(u) \geq 0,\ i = 1, \cdots, M$

Multiplicateurs de Lagrange

Soit $h : \mathbb{R}^d \to \mathbb{R}$

Problème : Minimiser $h(u)$ sous contraintes $g_i(u) \geq 0,\ i = 1, \cdots, M$

- Le Lagrangien :

$$L(u, \lambda) = h(u) - \sum_{i=1}^{M} \lambda_i g_i(u)$$

Les variables λ_i sont appelées **les variables duales**.

Multiplicateurs de Lagrange

Soit $h : \mathbb{R}^d \to \mathbb{R}$

Problème : Minimiser $h(u)$ sous contraintes $g_i(u) \geq 0,\ i = 1, \cdots, M$

- Le Lagrangien :

$$L(u, \lambda) = h(u) - \sum_{i=1}^{M} \lambda_i g_i(u)$$

Les variables λ_i sont appelées **les variables duales**.

Problème dual :

Minimiser $L(u, \lambda)$ par rapport à $u \Rightarrow u_\lambda$
Maximiser $\mathcal{L}(\lambda) = L(u_\lambda, \lambda)$ par rapport aux variables duales $\lambda_i \Rightarrow \lambda_i^*$

Multiplicateurs de Lagrange

Soit $h : \mathbb{R}^d \to \mathbb{R}$

Problème : Minimiser $h(u)$ sous contraintes $g_i(u) \geq 0,\ i = 1, \cdots, M$

- Le Lagrangien :

$$L(u, \lambda) = h(u) - \sum_{i=1}^{M} \lambda_i g_i(u)$$

Les variables λ_i sont appelées **les variables duales**.

Problème dual :

Minimiser $L(u, \lambda)$ par rapport à $u \Rightarrow u_\lambda$
Maximiser $\mathcal{L}(\lambda) = L(u_\lambda, \lambda)$ par rapport aux variables duales $\lambda_i \Rightarrow \lambda_i^*$

Condition de 1er ordre de Karush-Kuhn-Tuker (KKT): $\nabla_u L(u, \lambda) = 0$

$\frac{\partial h}{\partial u_k} - \sum_{i=1}^{M} \lambda_i \frac{\partial g_i}{\partial u_k} = 0,\ k = 1, \cdots, d$

Condition complémentaire de KKT : $\lambda_i^* g_i(u_{\lambda^*}) = 0$

Si $g_i(u_{\lambda^*}) > 0$, alors $\lambda_i^* = 0$

Problème dual de SVM

Le Lagrangien pour SVM :

$$L(w, b, \lambda) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{M} \lambda_i [y_i(w \cdot x_i + b) - 1]$$

Condition de 1er ordre de KKT

$$\begin{cases} \dfrac{\partial L}{\partial b} = -\sum_{i=1}^{M} \lambda_i y_i = 0 \Leftrightarrow \sum_{i=1}^{M} \lambda_i y_i = 0 \\ \dfrac{\partial L}{\partial w} = w - \sum_{i=1}^{M} \lambda_i y_i x_i = 0 \Leftrightarrow w = \sum_{i=1}^{M} \lambda_i y_i x_i \end{cases}$$

Problème dual de SVM

Le Lagrangien pour SVM :

$$L(w, b, \lambda) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{M} \lambda_i [y_i(w \cdot x_i + b) - 1]$$

Condition de 1er ordre de KKT

$$\begin{cases} \dfrac{\partial L}{\partial b} = -\sum_{i=1}^{M} \lambda_i y_i = 0 \Leftrightarrow \sum_{i=1}^{M} \lambda_i y_i = 0 \\ \dfrac{\partial L}{\partial w} = w - \sum_{i=1}^{M} \lambda_i y_i x_i = 0 \Leftrightarrow w = \sum_{i=1}^{M} \lambda_i y_i x_i \end{cases}$$

Problème dual :

$$\text{Maximiser } \mathcal{L}(\lambda) = \sum_{i=1}^{M} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{M} \lambda_i \lambda_j y_i y_j \langle x_i, x_j \rangle, \quad S.C.\ \lambda_i \geq 0 \text{ et } \sum_{i=1}^{M} \lambda_i y_i = 0$$

Problème dual de SVM

Le Lagrangien pour SVM :

$$L(w, b, \lambda) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{M} \lambda_i [y_i(w \cdot x_i + b) - 1]$$

Condition de 1er ordre de KKT

$$\begin{cases} \dfrac{\partial L}{\partial b} = -\sum_{i=1}^{M} \lambda_i y_i = 0 \Leftrightarrow \sum_{i=1}^{M} \lambda_i y_i = 0 \\ \dfrac{\partial L}{\partial w} = w - \sum_{i=1}^{M} \lambda_i y_i x_i = 0 \Leftrightarrow w = \sum_{i=1}^{M} \lambda_i y_i x_i \end{cases}$$

Problème dual :

$$\text{Maximiser } \mathcal{L}(\lambda) = \sum_{i=1}^{M} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{M} \lambda_i \lambda_j y_i y_j \langle x_i, x_j \rangle, \quad S.C.\ \lambda_i \geq 0 \text{ et } \sum_{i=1}^{M} \lambda_i y_i = 0$$

La solution du problème primal:

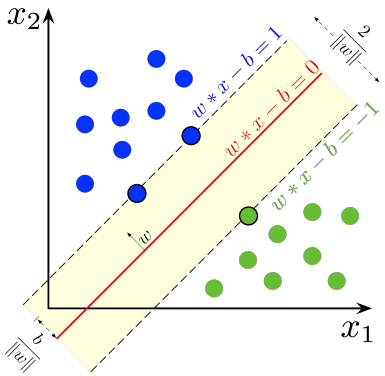
$w^* = \sum_{i=1}^{M} \lambda_i^* y_i x_i$

b^* : condition de normalisation pour un i.

Les vecteurs supports

Condition complémentaire de KKT : $\lambda_i^*[y_i(w^* \cdot x_i + b^*) - 1] = 0$

Le nombre de $\lambda_i^* > 0$ peut être petit



Les vecteurs supports Sont les points x_i tels que $\lambda_i^* > 0$, situés sur les frontières définissant la marge maximale : $y_i(w^* \cdot x_i + b^*) = 1$

Synthèse SVM linéaire

La fonction de décision est alors

$$D(x) = \mathbf{1}_{\{w^* \cdot x + b^* \geq 0\}} - \mathbf{1}_{\{w^* \cdot x + b^* < 0\}}$$

avec

- $w^* = \sum_{i=1}^{M} \lambda_i^* y_i x_i$
- b^* : condition de normalisation

ou encore

$$D(x) = \begin{cases} 1, \text{ si } \sum\limits_{x_i \in SV} (\lambda_i^* y_i \langle x_i, x \rangle + b^*) \geq 0 \\ \\ -1, \text{ si } \sum\limits_{x_i \in SV} (\lambda_i^* y_i \langle x_i, x \rangle + b^*) < 0 \end{cases}$$

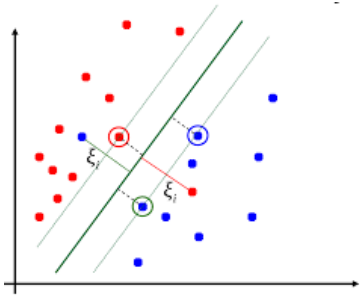
avec $\langle x_i, x \rangle = x_i \cdot x$.

SVM linéaire pour des données non séparables

Marge souple (soft margin)

- Variables ressorts ξ_i(slack variables)
- Contraintes relaxées

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0,\ i = 1, \cdots, M$$



- ξ_i renseigne à quel point l'exemple i ne respecte pas la contrainte
  - Si $\xi_i = 0$, l'exemple respecte la contrainte
  - Si $\xi_i > 1$, l'exemple est mal classé
  - Si $0 < \xi_i < 1$, l'exemple est bien classé mais franchit la marge

Les variables ξ_i risque de prendre de grandes valeurs

$\Rightarrow$ Pénaliser les grandes valeurs de ξ_i

SVM linéaire pour des données non séparables

Les variables ξ_i risque de prendre de grandes valeurs
$\Rightarrow$ Pénaliser les grandes valeurs de ξ_i

Nouveau problème primal : $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{M}\xi_i$

sous les contraintes $\begin{cases} y_i(w \cdot x_i + b) \geq 1 - \xi_i \\ \xi_i \geq 0 \end{cases}$

$C > 0$ paramètre, constante de tolérence à ajuster

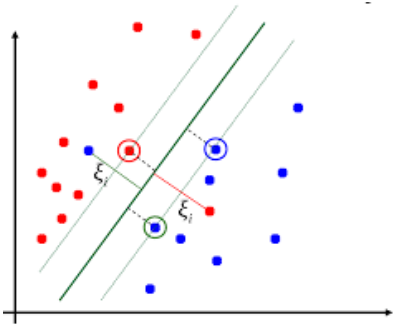
Problème dual :

$$\text{Maximiser } \mathcal{L}(\lambda) = \sum_{i=1}^{M}\lambda_i - \frac{1}{2}\sum_{i,j=1}^{M}\lambda_i\lambda_j y_i y_j \langle x_i, x_j\rangle, \quad S.C.\ 0 \leq \lambda_i \leq \mathbf{C} \text{ et } \sum_{\mathbf{i=1}}^{\mathbf{M}}\lambda_\mathbf{i}\mathbf{y_i} = \mathbf{0}$$

Vecteurs supports

Condition de KKT :

- $\lambda_i^*[y_i(w^* \cdot x_i + b^*) + \xi_i^* - 1] = 0$
- $\xi_i^*(\lambda_i^* - C) = 0$



Vecteurs supports Deux types :

- Les vecteurs correspondants à des variables ressorts nulles. Situés sur les frontières de la région définissant la marge : $\xi_i = 0$ et $0 < \lambda_i^* < C$
- Les vecteurs correspondants à des variables ressorts non nulles : $\xi_i^* > 0$ $(\lambda_i = C)$

Vecteurs non support : $\lambda_i^* = 0$ et $\xi_i^* = 0$

Synthèse SVM linéaire à marge souple

La fonction de décision est alors

$$D(x) = \mathbf{1}_{\{w^* \cdot x + b^* \geq 0\}} - \mathbf{1}_{\{w^* \cdot x + b^* < 0\}}$$

avec

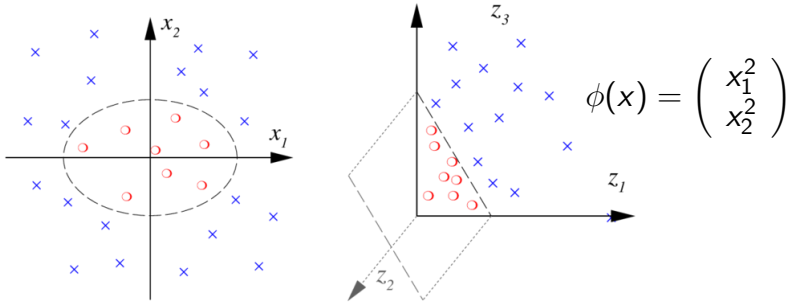
- $w^* = \sum_{i=1}^{M} \lambda_i^* y_i x_i$
- b^* : tel que $y_i(w^* \cdot x_i + b^*) = 1 - \xi_i$, $\forall x_i$, $0 < \lambda_i^* < C$

ou encore

$$D(x) = \begin{cases} 1, \text{ si } \sum\limits_{x_i \in SV} (\lambda_i^* y_i \langle x_i, x \rangle + b^*) \geq 0 \\ \\ -1, \text{ si } \sum\limits_{x_i \in SV} (\lambda_i^* y_i \langle x_i, x \rangle + b^*) < 0 \end{cases}$$

SVM à Noyau

- Données d'apprentissage sont intrinsèquement non séparables linéairement
- Plonger le problème non linéaire dans un espace de plus grande dimension (espace de représentation : feature space) dans lequel le problème devient linéaire
- Fonction de transformation ϕ



- La fonction noyau (kernel function)

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

SVM non linéaire

La fonction de décision est alors

$$D(x) = \mathbf{1}_{\{w^* \cdot \phi(x) + b^* \geq 0\}} - \mathbf{1}_{\{w^* \cdot \phi(x) + b^* < 0\}}$$

avec

- $w^* = \sum_{i=1}^{M} \lambda_i^* y_i \phi(x_i)$
- b^* : tel que $y_i(w^* \cdot \phi(x_i) + b^*) = 1 - \xi_i$, $\forall x_i$, $0 < \lambda_i^* < C$

ou encore

$$D(x) = \begin{cases} 1, \text{ si } \sum\limits_{x_i \in SV} (\lambda_i^* y_i \langle \phi(x_i), \phi(x) \rangle + b^*) \geq 0 \\ -1, \text{ si } \sum\limits_{x_i \in SV} (\lambda_i^* y_i \langle \phi(x_i), \phi(x) \rangle + b^*) < 0 \end{cases}$$

La fonction ϕ intervient uniquement dans des produits scalaires.

Astuce du noyau (kernel trick) : La connaissance de l'expression explicite de ϕ n'est pas nécessaire. Seule la connaissance de la fonction noyau k définie par $k(x, x') = \langle \phi(x), \phi(x') \rangle$ est requise.

Exemples de noyau

- Noyau linéaire
$$k(x, x') = \langle x, x' \rangle$$
- Noyau polynômiale de degré d
$$k(x, x') = (r + \gamma \langle x, x' \rangle)^d$$
- Noyau Gaussien (RBF)
$$k(x, x') = \exp(-\frac{\|x - x'\|^2}{2\sigma^2}) = \exp(-\gamma \|x - x'\|^2)$$
- Noyau sigmoïde
$$k(x, x') = \tanh(r + \gamma \langle x, x' \rangle)$$

Constructions de noyaux

- $k(x, y) = k_1(x, y) + k_2(x, y)$
- $k(x, y) = \alpha k_1(x, y)$
- $k(x, y) = k_1(x, y)k_2(x, y)$
- $k(x, y) = k_3(\phi(x), \phi(y))$
- $k(x, y) = xBy^T$ avec B une matrice $N \times N$ symétrique, semi-définie positive.

Cas multi-classes

Un contre tous OVR

- L'idée consiste simplement à transformer le problème à k classes en k classifieurs binaires.
- Le classement est donné par le classifieur qui répond le mieux.

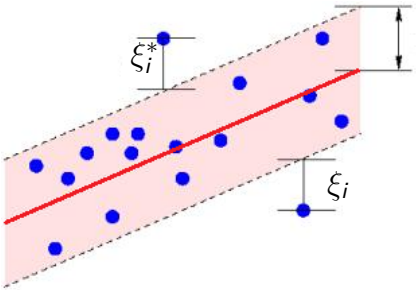
Un contre un OVO

- Le problème est transformé en $\frac{k(k-1)}{2}$ classifieurs binaires : chaque classe i étant en effet comparée à chaque classe j .
- Le classement est donné par le vote majoritaire...

SVM pour la régression

Régression

- La variable y prend des valeurs réels
- On cherche $f(x) = \langle w, \phi(x) \rangle + b$
  - aussi "plate" que possible
  - $\|f(x_i) - y_i| < \varepsilon$ pour tous les i.



ε-tube

- La condition d'aplatissement se traduit par la minimisation de $\|w\|^2 = \langle w, w \rangle$.
- On introduit les "variables d'assouplissement" ξ_t et ξ_t^*

SVM pour la régression

Problème primal : $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{M}(\xi_i + \xi_i^*)$

sous les contraintes $\begin{cases} y_i - (w \cdot \phi(x_i) + b) \leq \varepsilon + \xi_i \\ (w \cdot \phi(x_i) + b) - y_i \leq \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases}$

$C > 0$ paramètre, constante de tolérance à ajuster

Problème dual :

Maximiser $\mathcal{L}(\lambda) = \frac{1}{2}\sum_{i,j=1}^{M}(\lambda_i - \lambda_i^*)(\lambda_j - \lambda_j^*)K(x_i, x_j) + \varepsilon\sum_{i=1}^{M}(\lambda_i + \lambda_i^*) - \sum_{i=1}^{M} y_i(\lambda_i - \lambda_i^*),$

$S.C.\ 0 \leq \lambda_i, \lambda_i^* \leq \mathbf{C}$ et $\sum_{\mathbf{i=1}}^{\mathbf{M}}(\lambda_\mathbf{i} - \lambda_\mathbf{i}^*) = \mathbf{0}$

End lecture 4

Machine learning

Méthodes d'ensemble

M. Berrada

Ecole Nationale Supérieure des Arts et Métiers de Meknès
Université Moulay Ismail

2021-2022

Sommaire

1. Decision tree
   - Introduction
   - Algorithme ID3
   - Algorithme CART
   - Pruning
2. Méthodes d'ensembles parallèles
   - Bagging
   - Random Forest
3. Boosting (Méthodes séquentielles)
   - Boosting
   - Bagging vs Boosting

Sommaire

1. Decision tree
   - Introduction
   - Algorithme ID3
   - Algorithme CART
   - Pruning
2. Méthodes d'ensembles parallèles
   - Bagging
   - Random Forest
3. Boosting (Méthodes séquentielles)
   - Boosting
   - Bagging vs Boosting

Decision tree

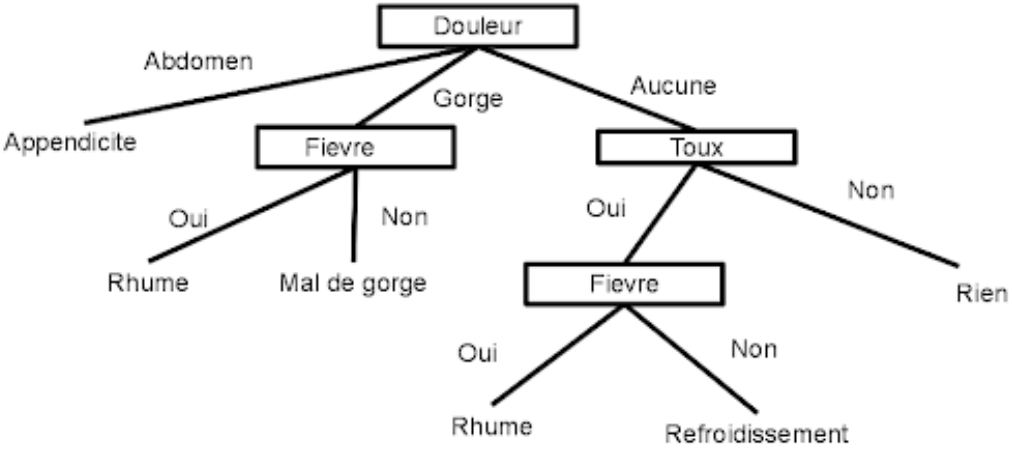
Arbre de décision

C'est une séquence de décision (tests) représentée sous la forme graphique d'un arbre. En théorie des graphes, un arbre est un graphe orienté acyclique et connexe.

Decision tree

Arbre de décision

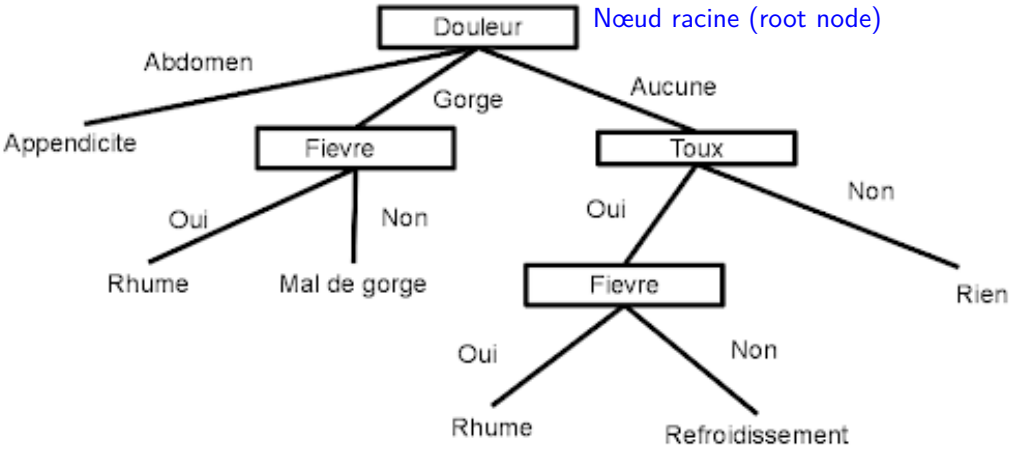
C'est une séquence de décision (tests) représentée sous la forme graphique d'un arbre. En théorie des graphes, un arbre est un graphe orienté acyclique et connexe.



Decision tree

Arbre de décision

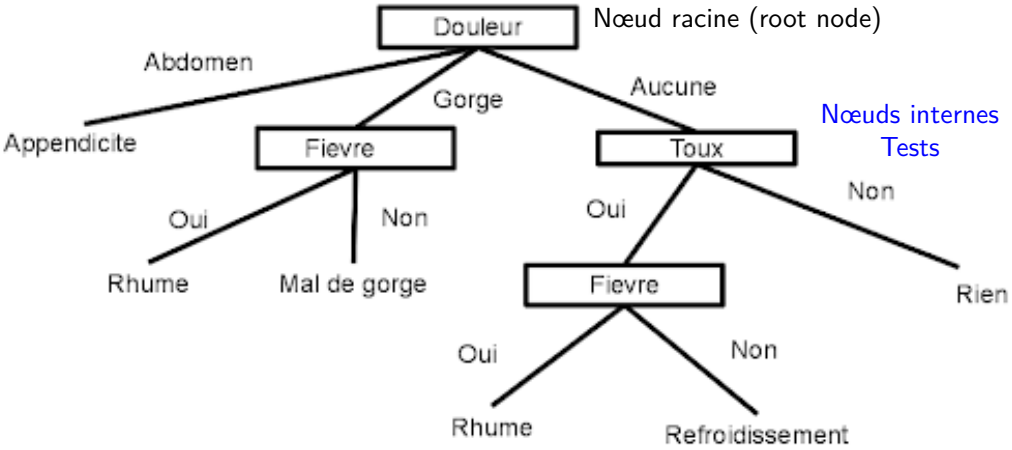
C'est une séquence de décision (tests) représentée sous la forme graphique d'un arbre. En théorie des graphes, un arbre est un graphe orienté acyclique et connexe.



Decision tree

Arbre de décision

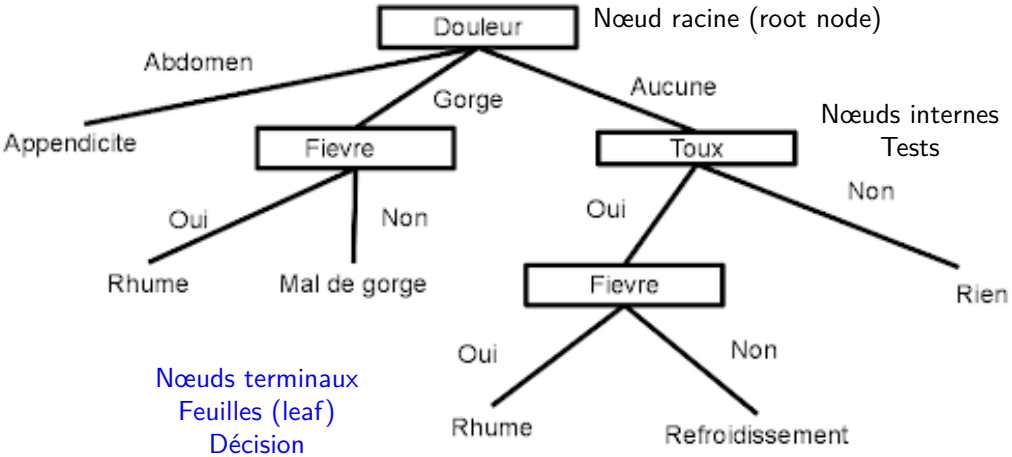
C'est une séquence de décision (tests) représentée sous la forme graphique d'un arbre. En théorie des graphes, un arbre est un graphe orienté acyclique et connexe.



Decision tree

Arbre de décision

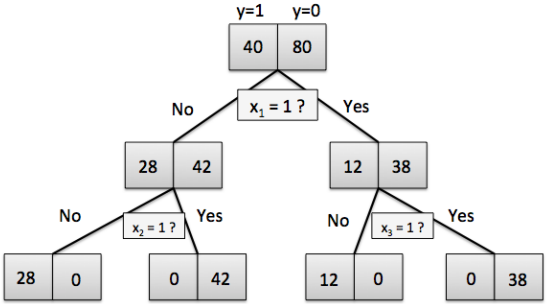
C'est une séquence de décision (tests) représentée sous la forme graphique d'un arbre. En théorie des graphes, un arbre est un graphe orienté acyclique et connexe.



Top-Down Induction

Ensemble d'apprentissage $\{(x_i, y_i),\ i = 1, \cdots, M\}$, $x_i \in \mathbb{R}^N$ $(x_i = (x_i^1, \cdots, x_i^N))$ les attributs et y_i la variable cible.

Top-Down Induction:

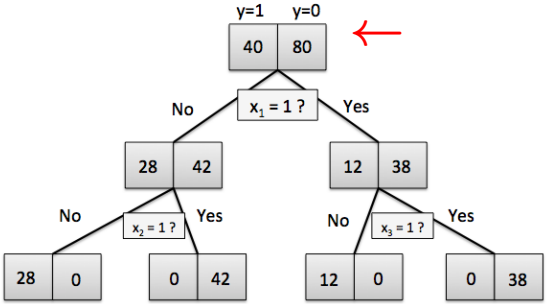


Top-Down Induction

Ensemble d'apprentissage $\{(x_i, y_i),\ i = 1, \cdots, M\}$, $x_i \in \mathbb{R}^N$ $(x_i = (x_i^1, \cdots, x_i^N))$ les attributs et y_i la variable cible.

Top-Down Induction:

- Au départ, tous les points sont placés dans le nœud racine

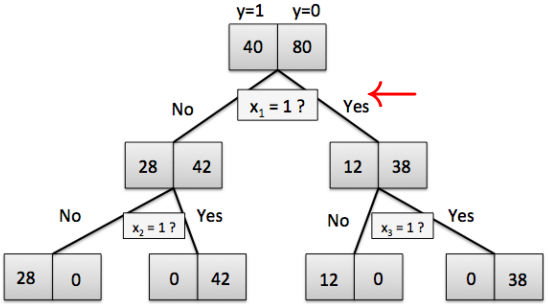


Top-Down Induction

Ensemble d'apprentissage $\{(x_i, y_i),\ i = 1, \cdots, M\}$, $x_i \in \mathbb{R}^N$ $(x_i = (x_i^1, \cdots, x_i^N))$ les attributs et y_i la variable cible.

Top-Down Induction:

- Au départ, tous les points sont placés dans le nœud racine
- Opération split : le nœud est coupé en K nœuds descendants selon l'attribut qui permet la meilleur séparation des données (maximise l'homogénéité).

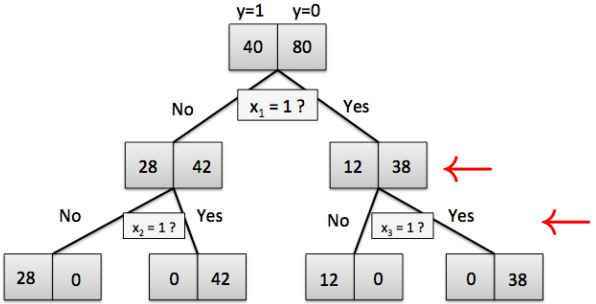


Top-Down Induction

Ensemble d'apprentissage $\{(x_i, y_i),\ i = 1, \cdots, M\}$, $x_i \in \mathbb{R}^N$ $(x_i = (x_i^1, \cdots, x_i^N))$ les attributs et y_i la variable cible.

Top-Down Induction:

- Au départ, tous les points sont placés dans le nœud racine
- Opération split : le nœud est coupé en K nœuds descendants selon l'attribut qui permet la meilleur séparation des données (maximise l'homogénéité).
- Le processus de split est ensuite répétée pour chaque sous-population (nœuds descendants) jusqu'à ce que le processus de séparation s'arrête.



Algorithmes

Comment choisir l'attribut qui maximise l'homogénéité ?

Comment traiter les attributs continus et les attributs qualitatifs?

Comment définir la taille optimale d'un arbre ? Un arbre trop profond est toujours synonyme de sur-apprentissage.

Il existe plusieurs algorithmes : ID3, C4.5, C5.0, **CART**

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

On suppose que la variable cible y est catégorielle (m catégories : classification)

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

On suppose que la variable cible y est catégorielle (m catégories : classification)

Entropie d'un nœud

- Les données du nœud S sont réparties en m groupes $C_1,\ C_2, \cdots, C_m$ selon la variable cible y. On note $p_i = \frac{|C_i|}{|S|}$

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

On suppose que la variable cible y est catégorielle (m catégories : classification)

Entropie d'un nœud

- Les données du nœud S sont réparties en m groupes $C_1,\ C_2, \cdots, C_m$ selon la variable cible y. On note $p_i = \frac{|C_i|}{|S|}$
- L'entropie de S est donnée par

$$H(S) = -\sum_{i=1}^{m} p_i \log(p_i)$$

Elle mesure l'écart de la distribution de y par rapport à celle uniforme :

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

On suppose que la variable cible y est catégorielle (m catégories : classification)

Entropie d'un nœud

- Les données du nœud S sont réparties en m groupes $C_1,\ C_2, \cdots, C_m$ selon la variable cible y. On note $p_i = \frac{|C_i|}{|S|}$
- L'entropie de S est donnée par

$$H(S) = -\sum_{i=1}^{m} p_i \log(p_i)$$

Elle mesure l'écart de la distribution de y par rapport à celle uniforme :

- $H(S) = 0$ nœud homogène (tous les $p_i = 0$ sauf un)
- $H(S) =$ max les C_i sont de même taille ($p_1 = p_2 = \cdots = p_m = 1/m$)

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

On suppose que la variable cible y est catégorielle (m catégories : classification)

Entropie d'un nœud

- Les données du nœud S sont réparties en m groupes $C_1,\ C_2, \cdots, C_m$ selon la variable cible y. On note $p_i = \frac{|C_i|}{|S|}$
- L'entropie de S est donnée par

$$H(S) = -\sum_{i=1}^{m} p_i \log(p_i)$$

Elle mesure l'écart de la distribution de y par rapport à celle uniforme :

- $H(S) = 0$ nœud homogène (tous les $p_i = 0$ sauf un)
- $H(S) =$ max les C_i sont de même taille ($p_1 = p_2 = \cdots = p_m = 1/m$)

Gain d'information sur un attribut catégoriel

- Les données du nœud S sont réparties en k groupes $S_1,\ S_2, \cdots, S_k$ selon l'attribut catégoriel a (k catégories). On note $p_i = \frac{|S_i|}{|S|}$.

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

On suppose que la variable cible y est catégorielle (m catégories : classification)

Entropie d'un nœud

- Les données du nœud S sont réparties en m groupes $C_1,\ C_2, \cdots, C_m$ selon la variable cible y. On note $p_i = \frac{|C_i|}{|S|}$
- L'entropie de S est donnée par

$$H(S) = -\sum_{i=1}^{m} p_i \log(p_i)$$

Elle mesure l'écart de la distribution de y par rapport à celle uniforme :

- $H(S) = 0$ nœud homogène (tous les $p_i = 0$ sauf un)
- $H(S) =$ max les C_i sont de même taille ($p_1 = p_2 = \cdots = p_m = 1/m$)

Gain d'information sur un attribut catégoriel

- Les données du nœud S sont réparties en k groupes $S_1,\ S_2, \cdots, S_k$ selon l'attribut catégoriel a (k catégories). On note $p_i = \frac{|S_i|}{|S|}$.
- Le gain d'information sur l'attribut a

$$GI(S, a) = H(S) - \sum_{i=1}^{k} p_i H(S_i)$$

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

- Calculer le GI de S pour chaque attribut pas encore utilisé

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

- Calculer le GI de S pour chaque attribut pas encore utilisé
- Choisir l'attribut a de GI maximal

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

- Calculer le GI de S pour chaque attribut pas encore utilisé
- Choisir l'attribut a de GI maximal
- Créer le test (décision) sur cet attribut dans le nœud S et générer les sous nœuds $S_1, \cdots, S_k$ correspondant à la partition sur l'attribut choisi a

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

- Calculer le GI de S pour chaque attribut pas encore utilisé
- Choisir l'attribut a de GI maximal
- Créer le test (décision) sur cet attribut dans le nœud S et générer les sous nœuds $S_1, \cdots, S_k$ correspondant à la partition sur l'attribut choisi a
- Répéter le processus sur les nouveaux nœuds

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

- Calculer le GI de S pour chaque attribut pas encore utilisé
- Choisir l'attribut a de GI maximal
- Créer le test (décision) sur cet attribut dans le nœud S et générer les sous nœuds $S_1, \cdots, S_k$ correspondant à la partition sur l'attribut choisi a
- Répéter le processus sur les nouveaux nœuds
- On arrête si
  - $H(S) = 0$, feuille
  - Pas d'attribut non utilisé : feuille sur la classe majoritaire
  - $S = \emptyset$, feuille sur la classe majoritaire du parent

Algorithme ID3 (Iterative Dichotomiser 3)

Algorithme : ID3

- Calculer le GI de S pour chaque attribut pas encore utilisé
- Choisir l'attribut a de GI maximal
- Créer le test (décision) sur cet attribut dans le nœud S et générer les sous nœuds $S_1, \cdots, S_k$ correspondant à la partition sur l'attribut choisi a
- Répéter le processus sur les nouveaux nœuds
- On arrête si
  - $H(S) = 0$, feuille
  - Pas d'attribut non utilisé : feuille sur la classe majoritaire
  - $S = \emptyset$, feuille sur la classe majoritaire du parent

L'algorithme C4.5 est une amélioration de ID3

Algorithme CART

Algorithme : CART (Classification And Regression Tree)

L'indice de Gini

Algorithme CART

Algorithme : CART (Classification And Regression Tree)

L'indice de Gini

- On pose $p_i = \frac{|C_i|}{|S|}$, l'indice de Gini de S

$$I(S) = \sum_{i=1}^{m} p_i(1 - p_i) = 1 - \sum_{i=1}^{m}(p_i)^2$$

Algorithme CART

Algorithme : CART (Classification And Regression Tree)

L'indice de Gini

- On pose $p_i = \frac{|C_i|}{|S|}$, l'indice de Gini de S

$$I(S) = \sum_{i=1}^{m} p_i(1 - p_i) = 1 - \sum_{i=1}^{m} (p_i)^2$$

Le nœud S est coupé en deux descendants suivant l'attribut j, seuil a_j (arbre binaire):

- Sous-nœud gauche S_g ($p_g = |S_g|/|S|$) (valeurs de l'attribut $v_j \leq a_j$),
- Sous-nœud droit S_d ($p_d = |S_d|/|S|$) (valeurs de l'attribut $v_j > a_j$),

Algorithme CART

Algorithme : CART (Classification And Regression Tree)

L'indice de Gini

- On pose $p_i = \frac{|C_i|}{|S|}$, l'indice de Gini de S

$$I(S) = \sum_{i=1}^{m} p_i(1 - p_i) = 1 - \sum_{i=1}^{m} (p_i)^2$$

Le nœud S est coupé en deux descendants suivant l'attribut j, seuil a_j (arbre binaire):

- Sous-nœud gauche S_g ($p_g = |S_g|/|S|$) (valeurs de l'attribut $v_j \leq a_j$),
- Sous-nœud droit S_d ($p_d = |S_d|/|S|$) (valeurs de l'attribut $v_j > a_j$),

Sélectionner l'attribut et le seuil qui minimisent l'impureté de S

$$(j^*, a_{j^*}) = \arg\min_{j, a_j} G(S, a)$$

où $G(S, a) = p_g I(S_g) + p_d I(S_d)$

Algorithme : CART (Classification And Regression Tree)

L'indice de Gini

- On pose $p_i = \frac{|C_i|}{|S|}$, l'indice de Gini de S

$$I(S) = \sum_{i=1}^{m} p_i(1 - p_i) = 1 - \sum_{i=1}^{m} (p_i)^2$$

Le nœud S est coupé en deux descendants suivant l'attribut j, seuil a_j (arbre binaire):

- Sous-nœud gauche S_g ($p_g = |S_g|/|S|$) (valeurs de l'attribut $v_j \leq a_j$),
- Sous-nœud droit S_d ($p_d = |S_d|/|S|$) (valeurs de l'attribut $v_j > a_j$),

Sélectionner l'attribut et le seuil qui minimisent l'impureté de S

$$(j^*, a_{j^*}) = \arg\min_{j, a_j} G(S, a)$$

où $G(S, a) = p_g I(S_g) + p_d I(S_d)$

On peut remplacer l'indice de Gini $I(S)$ par l'entropie $H(S)$.

Algorithme CART

Algorithme : CART (Classification And Regression Tree)

L'indice de Gini

- On pose $p_i = \frac{|C_i|}{|S|}$, l'indice de Gini de S

$$I(S) = \sum_{i=1}^{m} p_i(1 - p_i) = 1 - \sum_{i=1}^{m} (p_i)^2$$

Le nœud S est coupé en deux descendants suivant l'attribut j, seuil a_j (arbre binaire):

- Sous-nœud gauche S_g ($p_g = |S_g|/|S|$) (valeurs de l'attribut $v_j \leq a_j$),
- Sous-nœud droit S_d ($p_d = |S_d|/|S|$) (valeurs de l'attribut $v_j > a_j$),

Sélectionner l'attribut et le seuil qui minimisent l'impureté de S

$$(j^*, a_{j^*}) = \arg\min_{j, a_j} G(S, a)$$

où $G(S, a) = p_g I(S_g) + p_d I(S_d)$

On peut remplacer l'indice de Gini $I(S)$ par l'entropie $H(S)$.

Pour un problème de régression, on optimise le MSE

$$G(S, a) = p_g Var(S_g) + p_d Var(S_d)$$

Pruning (Élagage)

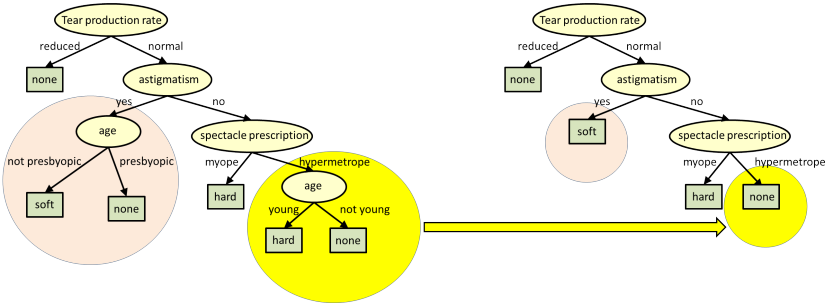
- Les arbres générés peuvent être de grande taille avec beaucoup de feuilles ⇒ sur-apprentissage

Pruning (Élagage)

- Les arbres générés peuvent être de grande taille avec beaucoup de feuilles ⇒ sur-apprentissage
- Pruning: réduire la taille des arbres de décision en supprimant des parties de l'arbre qui ne sont pas important

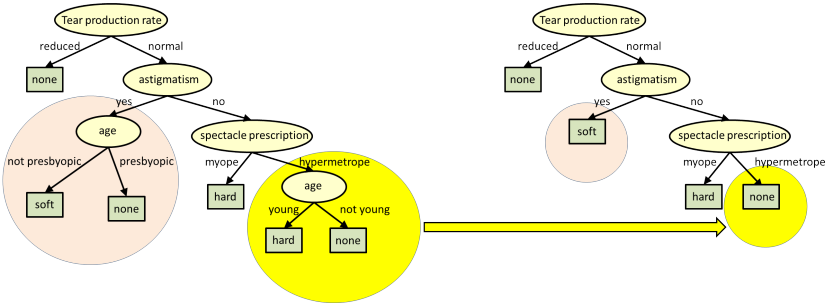
Pruning (Élagage)

- Les arbres générés peuvent être de grande taille avec beaucoup de feuilles ⇒ sur-apprentissage
- Pruning: réduire la taille des arbres de décision en supprimant des parties de l'arbre qui ne sont pas important



Pruning (Élagage)

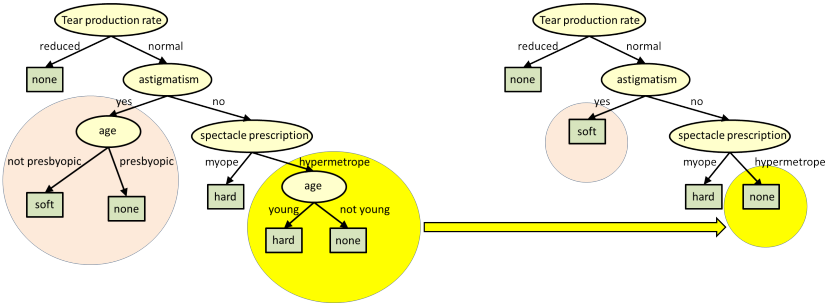
- Les arbres générés peuvent être de grande taille avec beaucoup de feuilles ⇒ sur-apprentissage
- Pruning: réduire la taille des arbres de décision en supprimant des parties de l'arbre qui ne sont pas important



- **Pre-pruning** remplace le critère d'arrêt (e.g. max. Tree depth or information gain (Attr)> minGain) par validation croisée. (Early stopping)

Pruning (Élagage)

- Les arbres générés peuvent être de grande taille avec beaucoup de feuilles ⇒ sur-apprentissage
- Pruning: réduire la taille des arbres de décision en supprimant des parties de l'arbre qui ne sont pas important



- **Pre-pruning** remplace le critère d'arrêt (e.g. max. Tree depth or information gain (Attr)> minGain) par validation croisée. (Early stopping)
- **Post-pruning** réduire la taille de l'arbre en remplaçant des noeuds internes et des sous-arbres par des feuilles. Ceci réduit la complexité et améliore la qualité de la classification. (e.g. Minimal Cost-Complexity Pruning)

Sommaire

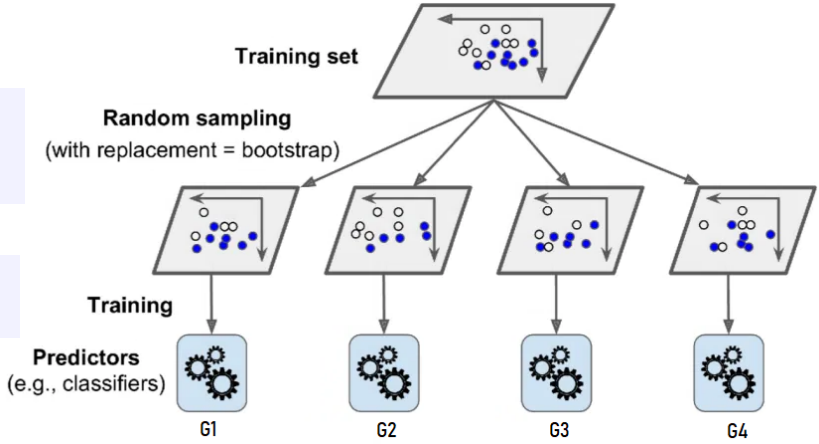
1. Decision tree
   - Introduction
   - Algorithme ID3
   - Algorithme CART
   - Pruning
2. Méthodes d'ensembles parallèles
   - Bagging
   - Random Forest
3. Boosting (Méthodes séquentielles)
   - Boosting
   - Bagging vs Boosting

Bagging

Bagging : Bootstrap Agregating

On tire au hasard B échantillons avec remise $E_i,\ i = 1, \cdots, B$ $(|E_i| = n)$

Pour chaque échantillon i on calcule le modèle $G_i(x)$



Régression : agrégation par la moyenne $G(x) = 1/B \sum_{i=1}^{B} G_i(x)$
Classement : agrégation par vote $G(x) = Votemajoritaire(G_1(x), \cdots, G_B(x))$.

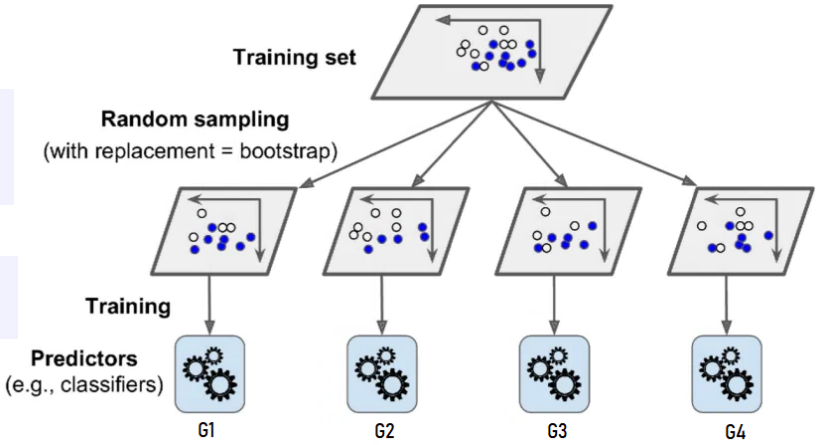
Le modèle obtenue est une moyenne. Il aide à la réduction de la variance à condition que les modèles G_i soient indépendants.

Bagging

Bagging : Bootstrap Agregating

On tire au hasard B échantillons avec remise $E_i,\ i = 1, \cdots, B\ (|E_i| = n)$

Pour chaque échantillon i on calcule le modèle $G_i(x)$



Régression : agrégation par la moyenne $G(x) = 1/B \sum_{i=1}^{B} G_i(x)$
Classement : agrégation par vote $G(x) = Votemajoritaire(G_1(x), \cdots, G_B(x))$.

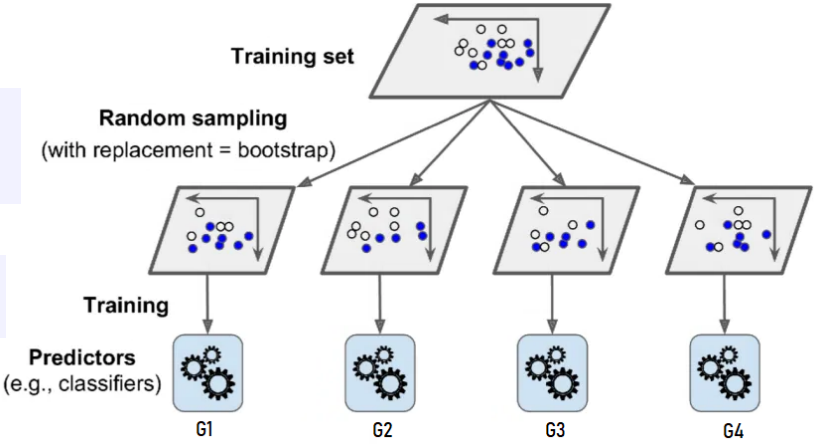
Le modèle obtenue est une moyenne. Il aide à la réduction de la variance à condition que les modèles G_i soient indépendants.

Bagging

Bagging : Bootstrap Agregating

On tire au hasard B échantillons avec remise $E_i,\ i = 1, \cdots, B$ $(|E_i| = n)$

Pour chaque échantillon i on calcule le modèle $G_i(x)$



Régression : agrégation par la moyenne $G(x) = 1/B \sum_{i=1}^{B} G_i(x)$
Classement : agrégation par vote $G(x) = Votemajoritaire(G_1(x), \cdots, G_B(x))$.

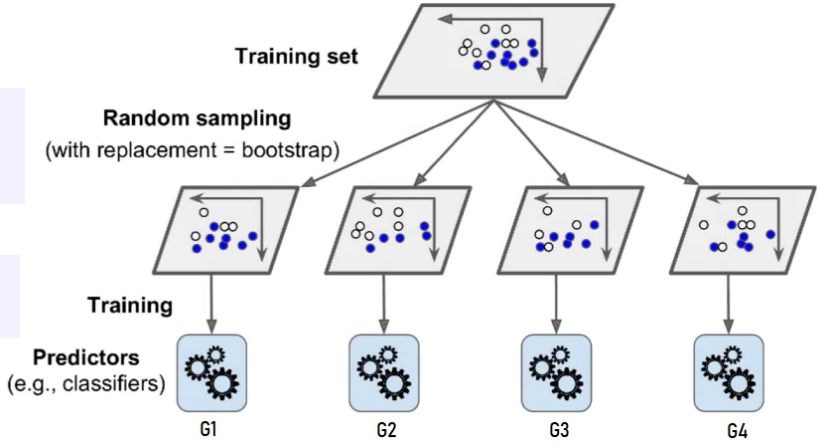
Le modèle obtenue est une moyenne. Il aide à la réduction de la variance à condition que les modèles G_i soient indépendants.

Bagging

Bagging : Bootstrap Agregating

On tire au hasard B échantillons avec remise $E_i,\ i = 1, \cdots, B$ $(|E_i| = n)$

Pour chaque échantillon i on calcule le modèle $G_i(x)$



Régression : agrégation par la moyenne $G(x) = 1/B \sum_{i=1}^{B} G_i(x)$
Classement : agrégation par vote $G(x) = Votemajoritaire(G_1(x), \cdots, G_B(x))$.

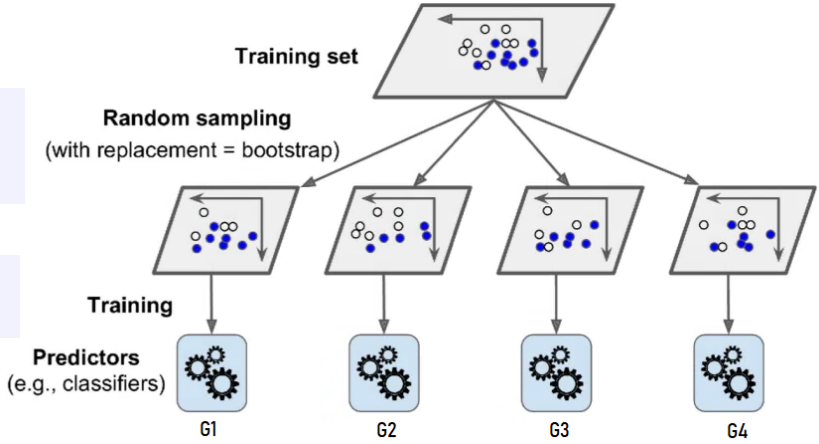
Le modèle obtenue est une moyenne. Il aide à la réduction de la variance à condition que les modèles G_i soient indépendants.

Bagging

Bagging : Bootstrap Agregating

On tire au hasard B échantillons avec remise $E_i,\ i = 1, \cdots, B$ $(|E_i| = n)$

Pour chaque échantillon i on calcule le modèle $G_i(x)$



Régression : agrégation par la moyenne $G(x) = 1/B \sum_{i=1}^{B} G_i(x)$
Classement : agrégation par vote $G(x) = Votemajoritaire(G_1(x), \cdots, G_B(x))$.

Le modèle obtenue est une moyenne. Il aide à la réduction de la variance à condition que les modèles G_i soient indépendants.

Les forêts aléatoires sont une amélioration du bagging

- Le modèle de base est celui des arbres de décision.
- Pour améliorer l'indépendance des modèles G_i, l'opération split d'un nœud est effectuée sur la base de q attributs tirés au hasard parmi les N attributs.

Sommaire

1. Decision tree
   - Introduction
   - Algorithme ID3
   - Algorithme CART
   - Pruning
2. Méthodes d'ensembles parallèles
   - Bagging
   - Random Forest
3. Boosting (Méthodes séquentielles)
   - Boosting
   - Bagging vs Boosting

Principe de **Boosting**

- Combiner les sorties de plusieurs classifieurs faibles (weak classifier) pour obtenir un résultat plus fort (strong classifier).

Principe de **Boosting**

- Combiner les sorties de plusieurs classifieurs faibles (weak classifier) pour obtenir un résultat plus fort (strong classifier).
- Un classifieur faible doit avoir un comportement de base un peu meilleur que l'aléatoire

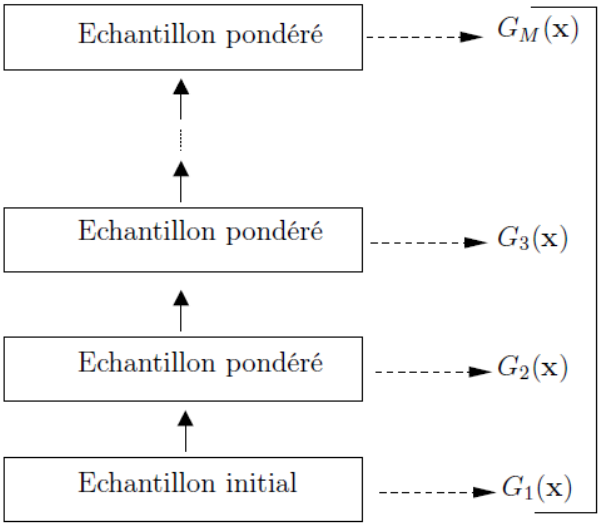
Principe de **Boosting**

- Combiner les sorties de plusieurs classifieurs faibles (weak classifier) pour obtenir un résultat plus fort (strong classifier).
- Un classifieur faible doit avoir un comportement de base un peu meilleur que l'aléatoire
- Chaque classifieur faible est pondéré par la qualité de sa classification : mieux il classe, plus il sera important.

Principe de **Boosting**

- Combiner les sorties de plusieurs classifieurs faibles (weak classifier) pour obtenir un résultat plus fort (strong classifier).
- Un classifieur faible doit avoir un comportement de base un peu meilleur que l'aléatoire
- Chaque classifieur faible est pondéré par la qualité de sa classification : mieux il classe, plus il sera important.
- Les exemples mal classés aurons un poids plus important (on dit qu'ils sont boostés) vis-à-vis de l'apprenant faible au prochain tour, afin qu'il pallie le manque.

Représentation de l'algorithme **AdaBoost**



$$G(\mathbf{x}) = \text{signe}\left[\sum_{m=1}^{M} \alpha_m G_m(\mathbf{x})\right]$$

Algorithme d'**AdaBoost**

Entrée :

- $\mathbf{x}$ l'observation à prévoir
- $d_n = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$ l'échantillon
- Une réglè faible
- M le nombre d'itérations.

1. Initialiser les poids $w_i = 1/n$, $i = 1, \ldots, n$
2. **Pour** $m = 1$ à M :
   (a) Ajuster la règle faible sur l'échantillon d_n pondéré par les poids $w_1, \ldots, w_n$, on note $g_m(\mathbf{x})$ l'estimateur issu de cet ajustement
   (b) Calculer le taux d'erreur :
   $$e_m = \frac{\sum_{i=1}^n w_i \mathbf{1}_{y_i \neq g_m(\mathbf{x}_i)}}{\sum_{i=1}^n w_i}.$$
   (c) Calculer : $\alpha_m = \log((1 - e_m)/e_m)$
   (d) Réajuster les poids :
   $$w_i = w_i \exp(\alpha_m \mathbf{1}_{y_i \neq g_m(\mathbf{x}_i)}), \quad i = 1, \ldots, n$$
3. **Sortie** : $\hat{g}_M(\mathbf{x}) = \sum_{m=1}^M \alpha_m g_m(\mathbf{x})$.

Bagging vs Boosting

Bagging on entraîne les modèles sur des sous-ensemble des données

- approche parallèle
- pas très efficace pour réduire le biais
- évite le sur-ajustement

Boosting chaque modèle cherche à corriger les faiblesses du précédent

- approche itérative
- utilise des modèles très simples
- réduit le biais
- risque de sur-ajustement

End lecture 5

Références

http://cedric.cnam.fr/vertigo/cours/ml2
https://en.wikipedia.org/
https://perso.univ-rennes2.fr/system/files/users/rouviere_l/poly_apprentissage.pdf